UNIVERSITAS GUNADARMA

FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI INFORMASI

PENULISAN ILMIAH

PROTOTIPE SISTEM MONITORING KUALITAS UDARA BERBASIS ESP32 DENGAN AKUISISI DATA PPM SENSOR MQ-7, MQ-135, DHT22 DAN VISUALISASI VIA BLYNK IOT.

Disusun Oleh :

NAMA	**:**	**Pedro Martin Sianipar**
NPM	**:**	**2112211**
PROGRAM STUDI	**:**	**Sistem Komputer**
PEMBIMBING	**:**	**Bintang Eka Putera, SKom, MMSI.**

JAKARTA

2025

PERNYATAAN ORISINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini,

Judul PI	: Prototipe Sistem Monitoring Kualitas Udara Berbasis ESP32 dengan Akuisisi Data PPM Sensor MQ-7, MQ-135, DHT22 dan Visualisasi via Blynk IoT.
Nama	: Pedro Martin Sianipar
NPM	: 21122111
Tanggal Sidang	: 1 Agustus 2025
Tanggal Lulus	: 1 Agustus 2025

Menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah, etika yang berlaku. Mengenai isi dan tulisan adalah merupakan tanggung jawab Penulis, bukan Universitas Gunadarma.

Demikian pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.

Diprint ukuran 3x4,Hitam

Jakarta, 17 Juli 2025

(Pedro Martin Sianipar)

LEMBAR PENGESAHAN

Judul Penelitian Ilmiah	: Prototipe Sistem Monitoring Kualitas Udara Berbasis ESP32 dengan Akuisisi Data PPM Sensor MQ-7, MQ-135, DHT22 dan Visualisasi via Blynk IoT
Nama	:Pedro Martin Sianipar
NPM	: 21122111
Program studi	: Sistem Komputer
Tanggal Sidang	:1 Agustus 2025
Tanggal Lulus	:1 Agustus 2025

Mengetahui,

Pembimbing	**Kepala subbagian Sidang PI**
(Bintang Eka Putera, SKom,MMSI)	**(Dr. Sri Nawangsari, SE., MM., MIKom)**

Ketua Program Studi

(Prof. Dr. Nur Sultan Salahuddin, SKom., MT.)

ABSTRAK

Pedro Martin Sianipar 21122111

PROTOTIPE SISTEM MONITORING KUALITAS UDARA BERBASIS ESP32 DENGAN AKUISISI DATA PPM SENSOR MQ-7, MQ-135, DHT22 DAN VISUALISASI VIA BLYNK IOT

Tulisan Ilmiah. Sistem Komputer Fakultas Ilmu Komputer dan Teknologi Informasi. Universitas Gunadarma. 2025

Kata kunci : informasi, kualitas, lingkungan, sensor, sistem, udara

(xii + 53 + Lampiran)

Penelitian ini bertujuan untuk merancang dan mengembangkan sistem monitoring kualitas udara secara real-time menggunakan mikrokontroler ESP32 dan tiga jenis sensor, yaitu sensor MQ-7, sensor MQ-135, dan sensor DHT22. Sistem ini dikembangkan untuk meningkatkan kesadaran akan kondisi lingkungan dan memberikan informasi kualitas udara yang akurat kepada pengguna secara berkelanjutan. Sensor MQ-7 digunakan untuk mendeteksi konsentrasi Karbon Monoksida (CO) di udara, yang nilainya dihitung dalam satuan Parts Per Million (PPM) melalui proses kalibrasi dan rumus spesifik dari datasheet. Sensor MQ-135 berfungsi untuk mengukur berbagai polutan udara umum, seperti Karbon Dioksida (CO2), yang juga dikonversi ke satuan PPM sebagai indikator kualitas udara. Sementara itu, sensor DHT22 digunakan untuk memantau suhu dan kelembaban lingkungan, menyediakan data kondisi atmosfer yang relevan. Data dari ketiga sensor ini divisualisasikan melalui tampilan LCD lokal dan dikirimkan secara real-time ke platform IoT Blynk untuk pemantauan jarak jauh. Sistem juga dilengkapi dengan buzzer yang akan aktif sebagai peringatan dini ketika konsentrasi polutan terukur melebihi ambang batas yang ditetapkan. Sistem ini diharapkan dapat membantu pengguna untuk memantau kualitas udara secara efisien, meningkatkan pemahaman tentang potensi bahaya polusi, dan mendukung terciptanya lingkungan yang lebih sehat dengan informasi yang mudah diakses dan respons yang cepat.

Daftar Pustaka (2005-2022)

KATA PENGANTAR

Puji dan Syukur dipanjatkan ke hadirat Allah SWT, atas berkah rahmat dan karunia-Nya, serta doa dan motivasi dari berbagai pihak sehingga dapat terselesaikannya tugas Penulisan Ilmiah yang berjudul "Prototipe Sistem Monitoring Kualitas Udara Berbasis ESP32 Dengan Akuisisi Data Ppm Sensor MQ-7, MQ-135, DHT22 Dan Visualisasi Via Blynk IoT" dengan sebaik-baiknya.

Dalam segala kerendahan hati, perkenankanlah ucapan terima kasih ini dihaturkan atas dorongan dan bantuan yang diterima, sehingga dapat menyelesaikan Penulisan Ilmiah ini, ucapan terima kasih dihaturkan kepada:

1. Prof. Dr. Hj. E. S. Margianti, SE, MM., selaku rektor Universitas Gunadarma.
2. Prof. Dr. Rer-nat Achmad Benny Mutiara QN.,Ssi., S.Kom., selaku Dekan Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
3. Dr. Nur Sultan Salahuddin, SKom., MT., selaku Ketua Program Studi Sistem Komputer Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma.
4. Dr. Sri Nawangsari, SE., MM., M.I.Kom., selaku Kepala Subbagian Sidang Penulisan Ilmiah Fakultas Ilmu Komputer dan Teknologi Informasi Universitas gunadarma.
5. Bapak Bintang Eka Putera, SKom,MMSI,, selaku dosen pembimbing yang telah banyak memberikan waktu dan sarannya selama pembuatan Penulisan Ilmiah ini.
6. Kedua Orang tua Bapak Thomson dan Ibu Delfina,berkat doa serta restu mereka terselesaikanlah Penulisan Ilmiah ini.
7. Teman-teman 3KB03 serta pihak lainnya yang secara langsung ikut memberikan dukungan.

Penulisan Ilmiah ini diharapkan dapat bermanfaat bagi aktivitas akademis Sistem Informasi Fakultas Ilmu Komputer dan Teknologi Informasi Universitas Gunadarma, dan menyadari atas segala keterbatasan pengetahuan dan kemampuan

diharapkan adanya kritik yang membangun dan semoga Penulisan Ilmiah ini dapat bermanfaat untuk orang banyak.

Jakarta, Juli 2025

(Pedro Martin Sianipar**)**

DAFTAR ISI

UNIVERSITAS GUNADARMA 1
PERNYATAAN ORISINALITAS DAN PUBLIKASI ii
LEMBAR PENGESAHAN iii
ABSTRAK iv
DAFTAR ISI vii
DAFTAR GAMBAR x
DAFTAR TABEL xi
DAFTAR LAMPIRAN xii
1.PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Ruang Lingkup 2
1.3 Tujuan Penelitian 3
1.4 Metode Penelitian 3
1.5 Sistematika Penulisan 4
2. TINJAUAN PUSTAKA 6
2.1 Kajian Penelitian 6
2.2 Sistem Pemantauan Kualitas Udara 8
2.3 Mikrokontroller ESP32 9
2.3.1 Spesifikasi ESP32 10
2.3.2 Pin-Pin Esp32 11
2.4 Sensor 13
2.4.1 Sensor MQ 14
2.4.2 Sensor DHT 16
2.5 Kabel Jumper 17
2.5.1 Jenis jenis Kabel Jumper 18
2.6 Bread Board 19
2.6.1 Jenis jenis Breadboard 19
2.6.2 Prinsip Kerja Breadboard 20
2.7. Buzzer 21
2.7.1 Pengertian Buzzer Elektronika 21

2.7.2 Prinsip Kerja Buzzer Elektronika 22
2.8 Arduino IDE 23
2.9 Blynk 24
2.10 Diagram Blok 25
2.11 Flowchart 26
2.11.1 Jenis Flowchart 27
2.11.2 Simbol pada flowchart 27
3. PEMBAHASAN 30
3.1. Blok Diagram 30
3.1.1 Blok Aktivator 30
3.1.2 Blok Input 30
3.1.3 Blok Proses 31
3.1.4 Blok Output 32
3.2 Analisa Rangkaian 33
3.3 Analisa Flowchart 35
3.4 Analisa Program 36
3.4.1 Definisi Awal dan Inisialisasi Pustaka 36
3.4.2 Kredensial dan Konfigurasi Hardware 37
3.4.3 Konstanta dan Variabel Global 38
3.4.4 Fungsi Pembantu Perhitungan Sensor MQ 40
3.4.5 Fungsi Kalibrasi Sensor MQ 41
3.4.6 Fungsi Kontrol Buzzer dari Blynk 42
3.4.7 Fungsi Pembacaan dan Pemrosesan Data Sensor 43
3.4.8 Fungsi Pengiriman Data ke Blynk 44
3.4.9 Fungsi Pembaruan Tampilan LCD 45
3.4.10 Fungsi Setup 45
3.4.11 Fungsi Loop 47
3.5 Pengujian Sensor 47
3.5.1 Sensor MQ135 dan MQ7 47
3.5.2 Sensor DHT22 49
3.5.3 Sensor MQ7 49

3.5.4 Sensor MQ135 50
4.PENUTUP 52
4.1 Kesimpulan 52
4.2 Saran 52
DAFTAR PUSTAKA 53
LAMPIRAN 1 ..... L-1

DAFTAR GAMBAR

Gambar 2. 1 ESP32 9
Gambar 2. 2 Pin pada ESP32..... 11
Gambar 2. 3 Sensor MQ 14
Gambar 2. 4 Sensor DHT22 16
Gambar 2. 5 Male to Male..... 18
Gambar 2. 6 Male to Female 18
Gambar 2. 7 Female to Female..... 18
Gambar 2. 8 Mini Breadboard..... 19
Gambar 2. 9 Medium Breadboard 20
Gambar 2. 10 Large Breadboard 20
Gambar 2. 11 Koneksivitas antara lubang Breadboard 20
Gambar 2. 12 Bentuk dan Simbol dari Buzzer..... 21
Gambar 2. 13 Logo Blynk 24
Gambar 2. 14 Simbol simbol pada Flowchart 28

Gambar 3. 1 Blok Diagram..... 30
Gambar 3. 2 Blok Aktivator 30
Gambar 3. 3 Blok Input 30
Gambar 3. 4 ESP32 yang Terhubung dengan Baseboard 31
Gambar 3. 5 Rangkaian pada Blok Output..... 32
Gambar 3. 6 Rangkaian Keseluruhan 33
Gambar 3. 7 Flowchart Prototipe Monitoring Udara Berbasis ESP32..... 35

DAFTAR TABEL

Tabel 2. 1 Tinjauan Penelitian 6
Tabel 2. 2 Spesifikasi ESP32 10
Tabel 2. 3 Pin Pada ESP32 11
Tabel 2. 4 Jenis Jenis MQ dengan Fungsinya 14
Tabel 2. 5 Spesifikasi DHT11 dan DHT22 16
Tabel 2. 6 Jenis Kabel Jumper 18
Tabel 2. 7 Jenis Breadboard 19

Tabel 3. 1 Definisi Awal dan Inisialisasi Pustaka 36
Tabel 3. 2 Kredensial dan Konfigurasi Hardware 37
Tabel 3. 3 Konstanta dan Variabel Global 38
Tabel 3. 4 Fungsi Pembantu Perhitungan Sensor MQ 40
Tabel 3. 5 Fungsi Kalibrasi Sensor MQ 41
Tabel 3. 6 Fungsi Kontrol Buzzer dari Blynk 42
Tabel 3. 7 Fungsi Pembacaan dan Pemrosesan Data Sensor 43
Tabel 3. 8 Fungsi Pengiriman Data ke Blynk 44
Tabel 3. 9 Fungsi Pembaruan Tampilan LCD 45
Tabel 3. 10 Fungsi Setup 45
Tabel 3. 11 Fungsi Loop 47
Tabel 3. 12 Pengujian Sensor MQ135 dan MQ7 47
Tabel 3. 13 Pengujian Sensor DHT22 49
Tabel 3. 14 Pengujian Sensor MQ7 49
Tabel 3. 15 Pengujian Sensor MQ135 50

DAFTAR LAMPIRAN

Lampiran 1 GAMBAR RANGKAIAN……...L-1

Lampiran 2 TAMPILAN BLYNK PADA WEB…….....L-2

Lampiran 3 TAMPILAN BLYNK PADA HANDPHONE …….L-3

Lampiran 4 GAMBAR ALAT ……...L-4

Lampiran 5 CODE ALAT ……..L-5

Lampiran 6 BIAYA PEMBUATAN ALAT……...L-11

1. PENDAHULUAN

1.1 Latar Belakang

Kualitas udara merupakan salah satu faktor krusial yang secara langsung memengaruhi kesehatan manusia dan ekosistem. Peningkatan aktivitas industri, urbanisasi, dan kepadatan lalu lintas telah berkontribusi pada lonjakan konsentrasi polutan di atmosfer, yang berdampak serius pada kesehatan pernapasan, kardiovaskular, dan sistem saraf manusia. Organisasi Kesehatan Dunia (WHO) melaporkan bahwa polusi udara merupakan salah satu risiko lingkungan terbesar bagi kesehatan, menyebabkan jutaan kematian prematur setiap tahunnya. Oleh karena itu, pemantauan kualitas udara secara akultif dan real-time menjadi sangat penting untuk identifikasi dini potensi bahaya dan pengambilan keputusan yang tepat guna mitigasi risiko.

Sistem pemantauan kualitas udara konvensional seringkali memerlukan biaya yang mahal, instalasi yang rumit, dan mobilitas yang terbatas, sehingga cakupan pemantauannya menjadi kurang optimal terutama di area yang luas atau terpencil. Seiring dengan perkembangan teknologi *Internet of Things* (IoT), muncul peluang untuk mengembangkan solusi pemantauan kualitas udara yang lebih efisien, terjangkau, dan dapat disebarkan secara luas. Dengan IoT memungkinkan pengumpulan data secara nirkabel.

Dalam konteks ini, mikrokontroler ESP32 menonjol sebagai pilihan yang ideal berkat kapabilitas Wi-Fi terintegrasinya, kinerja tinggi, dan konsumsi daya yang efisien, menjadikannya platform yang kuat untuk aplikasi IoT. Kombinasi sensor MQ135 untuk deteksi gas berbahaya seperti CO2, NH3, dan asap, serta sensor MQ7 yang spesifik untuk karbon monoksida (CO) gas yang tidak berbau, tidak berwarna, namun sangat mematikan memungkinkan analisis komprehensif terhadap berbagai polutan udara. Selain itu, integrasi sensor DHT22 memberikan data suhu dan kelembaban, yang tidak hanya memengaruhi kenyamanan termal tetapi juga relevan dalam interpretasi kualitas udara, mengingat kinerja sensor gas dapat dipengaruhi oleh kondisi lingkungan.

Untuk memastikan akses informasi yang mudah dan cepat, sistem ini mengintegrasikan LCD (*Liquid Crystal Display*) sebagai antarmuka tampilan lokal yang intuitif, memungkinkan pengguna memantau data secara langsung di lokasi. Lebih lanjut, pemanfaatan platform Blynk menghadirkan kemampuan pemantauan jarak jauh dan notifikasi real-time melalui aplikasi smartphone dengan keluaran buzzer Ketika polutan melebihi ambang batas. Ini memungkinkan pengguna untuk menerima peringatan dini jika tingkat polutan mencapai ambang batas berbahaya, sehingga tindakan pencegahan dapat segera diambil.

Berdasarkan urgensi kebutuhan akan informasi kualitas udara yang akurat dan mudah diakses, serta potensi besar yang ditawarkan oleh teknologi IoT, penelitian ini bertujuan untuk merancang dan mengimplementasikan sistem pemantauan kualitas udara berbasis IoT menggunakan ESP32 yang terintegrasi dengan sensor MQ135, MQ7, DHT22, serta dilengkapi dengan tampilan LCD dan notifikasi real-time melalui platform Blynk dengan alarm buzzer. Sistem ini diharapkan dapat menjadi solusi yang efektif dan terjangkau untuk memantau dan memberikan peringatan dini mengenai kondisi kualitas udara, berkontribusi pada lingkungan hidup yang lebih sehat dan aman.

1.2 Ruang Lingkup

Dengan munculnya permasalahan-permasalahan diatas dan untuk menghindari Analisa yang berkepanjangan dan luasnya ruang lingkup maka pembahasan ini di batasi dengan :

1. Zat yang dipantau yaitu gas CO,CO2 dan Suhu ruangan
2. Sensor gas yang digunakan sensor MQ-135, MQ-7, dan DHT22
3. Kontroler yang digunakan ESP32 dengan output Notifikasi Blynk, LCD, dan Buzzer
4. Harus tersambung di area yang ada jangkauan WiFi.

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah untuk merancang dan mengimplementasikan sebuah sistem monitoring kualitas udara berbasis IoT yang mampu mendeteksi dan menampilkan kondisi lingkungan udara secara real-time. Sistem ini diharapkan dapat meningkatkan kewaspadaan Masyarakat terhadap bahayanya Polutan pada lingkungan

1.4 Metode Penelitian

Metode penelitian ini akan menggunakan pendekatan kuantitatif dengan metode eksperimen untuk merancang, mengimplementasikan, dan menguji sistem pemantauan kualitas udara. Pendekatan ini dipilih untuk mengukur performa sistem dan memverifikasi fungsionalitasnya berdasarkan data yang terukur dari sensor.

Pada tahapan awal ini, penulis akan mengumpulkan dan meninjau berbagai sumber informasi relevan, seperti jurnal ilmiah, buku teks, dan artikel. Fokus utamanya adalah memahami prinsip kerja sistem otomatisasi berbasis mikrokontroler, serta mendalami karakteristik dan aplikasi spesifik dari sensor-sensor yang akan digunakan dalam proyek ini.

Setelah memperoleh Pemahaman Teoritis, Penelitian dilanjutkan dengan Membuat rancangan skematik sistem menggunakan software simulasi seperti Cirkit, serta menyusun logika kerja sistem dan program berdasarkan fungsi masing-masing sensor dan aktuator. Setelah Membuat Rancangan skematik, penulis merakit komponen elektronik di atas breadboard dan mengunggah program ke mikrokontroler ESP32 menggunakan Arduino IDE.

Tahap terakhir Penelitian adalah pengujian menyeluruh terhadap alat yang telah dibuat. Kami akan menguji sistem dalam berbagai skenario dan kondisi lingkungan untuk memastikan bahwa seluruh fungsi berjalan sesuai harapan. Berdasarkan hasil pengujian, evaluasi akan dilakukan untuk mengidentifikasi kelebihan dan kekurangan sistem, serta melakukan perbaikan yang diperlukan guna meningkatkan performa dan stabilitas.

1.5 Sistematika Penulisan

Sistematika penulisan laporan tugas akhir ini disusun agar pembaca dapat memahami alur penelitian secara logis dan terstruktur, Pendahuluan, yang berisi penjelasan mengenai latar belakang, ruang lingkup, batasan, tujuan, dan metode penelitian, serta sistematika penulisan itu sendiri. Selanjutnya, Tinjauan Pustaka membahas teori-teori relevan yang diperoleh dari berbagai sumber seperti jurnal, buku, dan artikel online.Setelah itu, laporan masuk ke dalam bab Pembahasan, di mana dijelaskan secara detail mengenai diagram blok dan cara kerja setiap komponen, termasuk alur proses (flowchart) dan program yang digunakan. Di bab ini, dilakukan juga uji coba terhadap alat yang telah dibuat untuk memastikan fungsinya berjalan sesuai harapan. Terakhir, bab Penutup berisi kesimpulan yang didapat dari hasil uji coba tersebut, serta saran-saran untuk pengembangan alat di masa mendatang agar lebih sempurna dan memberikan manfaat yang lebih besar

2. TINJAUAN PUSTAKA

2.1 Kajian Penelitian

Tabel 2. 1 Tinjauan Penelitian

No	Tahun	Peneliti dan Sumber	Judul Penelitian	Metoda Usulan	Hasil Penelitian
1	2018	Tri Fidrian Arya, Mahar Faiqurahman, dan Yufis Azhar	APLIKASI WIRELESS SENSOR NETWORK UNTUK SISTEM MONITORING DAN KLASIFIKASI KUALITAS UDARA	Wireless Sensor Network (WSN) untuk sistem pemantauan dan klasifikasi kualitas udara secara online, di mana sistem ini dirancang dengan arsitektur yang terdiri dari node sensor, sink node, dan komputer server dalam topologi star. Node sensor (menggunakan Arduino) dilengkapi sensor MQ-7 untuk CO, MQ-131 untuk O3, dan GP2Y1010 AU0F	buffer sebanyak 32-33 jumlah dapat memaksimalkan sumber daya SRAM pada Arduino, meskipun sisa SRAM kurang dari 200 byte dapat menyebabkan kegagalan sistem. Jangkauan komunikasi modul LoRa ditemukan stabil hingga 200 meter, namun pengiriman data menjadi tidak stabil antara 200-220 meter dan gagal pada jarak ≥ 240 meter, yang mana berbeda dengan klaim datasheet LoRa. Dalam penanganan kehilangan data,

2	2018	Jacquline Waworundeng, dan Oktoverano Lengkong	Sistem Monitoringdan Notifikasi Kualitas Udara dalam Ruangan dengan PlatformIoT	Penelitian ini mengusulkan pengembangan prototipe sistem monitoring dan notifikasi kualitas udara dalam ruangan menggunakan mikrokontroler Wemos dan sensor MQ135 yang terintegrasi dengan platform Internet of Things (IoT). Sensor MQ135 mendeteksi gas berbahaya seperti Amonia, Nitrogen Oksida, Alkohol, Benzena, Karbon Monoksida, dan Karbon Dioksida, lalu mengirimkan sinyal input ke	Hasil pengujian fungsionalitas sistem menunjukkan bahwa sensor MQ135 berhasil mendeteksi peningkatan level alkohol, CO, atau CO2. Wemos board berhasil menerima dan memproses sinyal input dari sensor. Blynk dapat menerima pesan notifikasi mengenai kualitas udara, sementara Thingspeakberhasil menerima dan menampilkan data sensor dalam bentuk grafik real-time. Berdasarkan pengujian selama 20 menit di dalam ruangan, nilai analog sensor rata-rata adalah 489,3, yang mengindikasikan bahwa kualitas udara berada pada level "medium polluted" (kisaran 400-800). Notifikasi yang diterima pengguna melalui Blynk Apps mengonfirmasi bahwa level kualitas udara terdeteksi sebagai

				mikrokontroler Wemos. Modul WiFi bawaan pada Wemos mengirimkan nilai sensor ke Thingspeak, yang berfungsi sebagai sistem monitoring dengan merekam data dalam bentuk grafi	"medium". Dengan demikian, sistem monitoring dan notifikasi kualitas udara yang dikembangkan berfungsi sesuai tujuan dan berpotensi untuk meningkatkan kesadaran akan pentingnya kualitas udara yang sehat dalam ruangan
3	2022	Ivanno Alexander Rombang, Lukas Bambang Setyawan, Gunawan Dewantoro	Perancangan Prototipe Alat Deteksi Asap Rokok dengan Sistem PurifierMenggunakan Sensor MQ-135 dan MQ-2	Penelitian ini mengusulkan perancangan prototipe alat deteksi asap rokok dengan sistem purifier berbasis Arduino Uno R3, menggunakan sensor MQ-135 untuk mendeteksi gas CO2 dan sensor MQ-2 untuk mendeteksi gas CO. Sistem purifier ini dilengkapi	Hasil pengujian menunjukkan bahwa prototipe alat deteksi asap rokok ini mampu membersihkan kualitas udara dalam ruangan prototipe. Rata-rata waktu pembersihan untuk kategori polusi sedikit adalah 34,52 menit (dengan kadar CO2 setelah pembersihan rata-rata 1055 ppm dan CO rata-rata 19 ppm). Untuk kategori sedang, rata-rata waktu pembersihan adalah 40,4 menit (dengan kadar CO2 setelah

				dengan exhaust fan dan filter karbon aktif untuk membersihkan udara yang tercemar asap rokok. Alat akan menampilkan hasil deteksi gas pada LCD I2C 16x2 dalam satuan ppm. Jika sensor mendeteksi kadar gas	pembersihan rata-rata 677 ppm dan CO rata-rata 19 ppm). Sementara itu, untuk kategori banyak, rata-rata waktu pembersihan adalah 58,3 menit (dengan kadar CO2 setelah pembersihan rata-rata 861 ppm dan CO rata-rata 19 ppm). Pengujian sensor MQ-135 dan MQ-2 menunjukkan respons yang baik terhadap asap rokok, dengan MQ-135 mendeteksi

2.2 Sistem Pemantauan Kualitas Udara

Lingkungan yang sehat sangatlah berpengaruh pada Kesehatan baik fisik maupun mental. Faktor penting yang menunjang lingkungan yang sehat adalah kualitas udara yang sehat. Udara mengandung oksigen yang dibutuhkan untuk hidup. Namun selain oksigen, terdapat zat lainnya dalam udara seperti karbon monoksida, karbon dioksida, formaldehid, jamur, virus, bakteri, debu dan sebagainya. Oksigen di dalam maupun di luar ruangan dapat terkontaminasi dengan zat-zat yang berbahaya bagi kesehatan. Dalam batasan tertentu kadar zat-zat tersebut masih dapat dinetralisir namun jika melampaui batas normal maka dapat mengganggu kesehatan. *World Health Organization*(WHO) menyatakan bahwa terdapat zat berbahaya yang berasal dari bangunan, material konstruksi, peralatan, proses pembakaran atau pemanasan dapat memicu masalah Kesehatan

Menurut Jacquline Waworundeng dan Oktoverano Lengkong pada penelitiannya dijelaskan Peningkatan aktivitas manusia telah memicu masalah pencemaran udara, sehingga dibutuhkan solusi untuk dapat meminimalisir efek yang dapat mengganggu kesehatan. Walaupun pada saat tertentu manusia dapat menggunakan indera untuk memperkirakan jika udara di lingkungan sekitarnya berada pada level normal dan tidak tercemar ataupun sebaliknya, namun untuk melakukan pemantauan secara terus menerus, manusia dibatasi oleh ruang dan waktu.Untuk melakukan pemantauan secara real-time dan mendapatkan data mengenai kualitas udara dapat dilakukan dengan membangun suatu perangkat keras yang terhubung dengan sistempemantauan kualitas udara

2.3 Mikrokontroller ESP32



Gambar 2. 1 ESP32

ESP32 adalah mikrokontroler dual-core yang mendukung konektivitas Wi-Fi dan Bluetooth. Mikrokontroler ini memiliki performa tinggi dengan konsumsi daya yang rendah, sangat cocok digunakan dalam sistem kendali dan monitoring berbasis IoT. ESP32 juga memiliki banyak GPIO dan mendukung berbagai komunikasi seperti UART, I2C, dan SPI yang memudahkan integrasi dengan sensor dan modul lain. Modul ESP32 Devkit V1 digunakan sebagai mikrokontroler utama dalam sistem. Perangkat ini mendukung komunikasi nirkabel seperti Wi-Fi dan Bluetooth, serta memiliki banyak pin I/O untuk integrasi sensor dan aktuator.

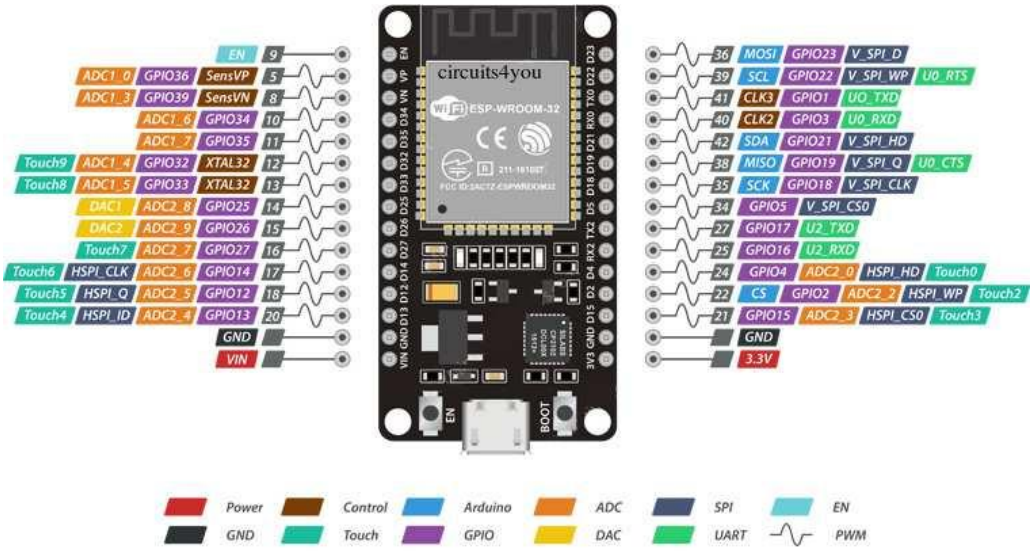
2.3.1 Spesifikasi ESP32

Tabel 2. 2 Spesifikasi ESP32

No	Spesifikasi	Keterangan
1	Tipe Prosesor	Dual-core Tensilica LX6
2	Kecepatan Prosesor	Hingga 240 MHz
3	Memori Flash	4 MB (tergantung modul, bisa 8MB atau 16MB)
4	RAM Internal	520 KB SRAM
5	Wi-Fi	IEEE 802.11 b/g/n, mendukung mode AP, STA, dan AP+STA
6	Bluetooth	Bluetooth v4.2 BR/EDR dan BLE
7	GPIO	Sekitar 30 pin input/output digital (tergantung jenis modul)
8	ADC	12-bit, hingga 18 channel
9	DAC	2 channel (8-bit Digital to Analog Converter)
10	Antarmuka Komunikasi	UART, SPI, I2C, CAN, PWM, I2S
11	Tegangan Operasional	3.0V – 3.6V (umumnya 3.3V)
12	Konsumsi Daya	Rendah (mendukung mode sleep dan deep sleep)
13	Ukuran Modul	± 25.5 mm x 18 mm (tergantung varian seperti DevKit V1, WROOM, dll)
14	Platform Pemrograman	Arduino IDE, MicroPython, ESP-IDF

2.3.2 Pin-Pin Esp32

ESP32 memiliki berbagai jenis pin yang digunakan untuk menghubungkan dan mengontrol berbagai perangkat input dan output.



Gambar 2. 2 Pin pada ESP32

Distribusi pin pada ESP32 mencakup pin daya, pin digital I/O, ADC, serta pin komunikasi seperti UART, SPI, dan I2C. Pengetahuan mengenai distribusi pin sangat penting untuk proses perancangan sistem.

Berikut merupakan penjelasan masing-masing jenis pin pada ESP32 :

Tabel 2. 3 Pin Pada ESP32

NO	Tipe PIN	Jenis Jenisnya	Kegunaan
1	Powers Pin (Daya)	3V3	Output 3.3V dari regulator internal, digunakan untuk memberi daya ke sensor/modul 3.3V.
		GND	Ground (negatif), harus terhubung ke semua komponen.
		5 V atau VIN	Input daya dari USB atau sumber eksternal 5V, juga bisa

			digunakan untuk memberi daya ke perangkat 5V lainnya.
2	Input Only PIN	GPIO36 (VP)	Input analog. Tidak bisa digunakan sebagai output atau PWM.
		GPIO39 (VN)	Input analog. Sering digunakan untuk sensor.
		GPIO34	Input digital/analog saja. Tidak bisa jadi output
		GPIO35	Input digital/analog saja. Tidak bisa jadi output
3	Digital I/O Pins	GPIO0	Boot pin. LOW saat flash. Hindari sebagai output awal.
		GPIO2	Aman digunakan. Bisa untuk LED, input, PWM, dll
		GPIO4	Serbaguna. Bisa PWM, ADC, digital I/O
		GPIO5	Biasa digunakan untuk Chip Select (SPI). Juga PWM/digital output
		GPIO12	Sensitif saat boot. Jangan ditarik HIGH saat menyala
		GPIO13–19, 21–23, 25–27, 32–33	Umumnya aman untuk input/output digital, PWM, dan ADC
		GPIO15	Boot sensitive. Harus LOW saat startup.

4	PWM (Pulse Width Modulation) Pins	GPIO2, 4, 5, 12–19, 21–23, 25–27, 32–33	Digunakan untuk servo motor, LED brightness, buzzer, dll.
5	ADC Pins (Analog to Digital Converter)	GPIO32–39	Input analog 12-bit
		GPIO34–39	Input-only, tidak bisa jadi output
6	UART (Serial Communication)	UART0	TX GPIO 1 = RX GPIO 3
		UART1	TX GPIO 10 = RX GPIO 9
		UART2	TX GPIO 17 = RX GPIO 16
7	PIN Khusus	GPIO 0	Digunakan saat flashing. Jangan digunakan untuk output LED saat boot
		GPIO 2	Aman untuk output, default low saat boot
		GPIO 15	Harus low saat booting (hindari untuk pull-up di awal).
		GPIO 34–39	Input only – tidak bisa digunakan untuk output atau PWM

2.4 Sensor

Menurut Susilo, T. dan Sari, R. (2018) dalam Jurnal Teknik Elektro dan Komputer "Sensor merupakan komponen utama dalam sistem akuisisi data yang berfungsi untuk menangkap besaran fisik seperti suhu, tekanan, kelembaban, atau gas, dan mengubahnya menjadi data digital yang bisa diproses lebih lanjut." Sensor merupakan merupakan device yang berfungsi untuk mengubah besaran mekanis,

magnetis, panas, sinar, dan kimia menjadi besaran listrik berupa tegangan, resistansi, atau arus listrik. Perubahan inilah yang dapat digunakan sebagai pembacaan pada rangkaian mikrokontroler.

2.4.1 Sensor MQ



Gambar 2. 3 Sensor MQ

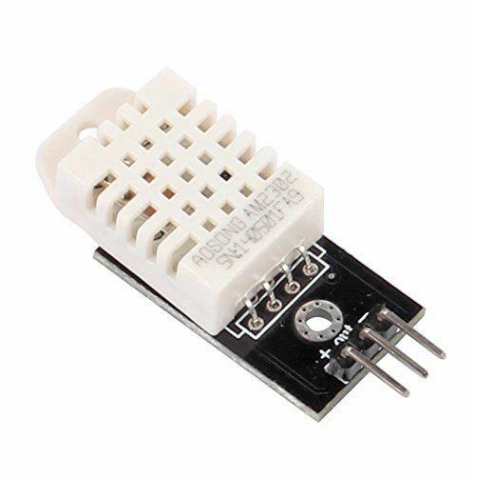
Sensor MQ adalah jenis sensor gas semikonduktor yang umum digunakan untuk mendeteksi berbagai konsentrasi gas. Sensor ini memiliki pin keluaran digital dan analog. Ketika tingkat gas di udara melampaui batas ambang tertentu, pin digital akan menjadi HIGH, dan pin keluaran analog akan mengeluarkan tegangan analog yang dapat digunakan untuk memperkirakan tingkat gas di udara. Sensor seri MQ dikenal memiliki kepekaan, waktu respons, dan pengukuran yang cepat dan tepat. Mereka bekerja berdasarkan perubahan nilai resistansi pada bahan penginderaan sensor ketika terjadi kontak dengan gas yang terdeteksi. Beberapa contoh sensor MQ diantaranya adalah

Tabel 2. 4 Jenis Jenis MQ dengan Fungsinya

NO	Nama Sensor	Fungsi
1	MQ-2	sensor gas yang sensitif terhadap bau metana, butana, LPG dan asap rokok
2	MQ-3	sensor gas yang sensitif terhadap bau alkohol, etanol dan asap rokok
3	MQ-4	sensor gas yang sensitif terhadap bau metana dan CNG

4	MQ-6	sensor gas yang sensitif terhadap bau gas LPG dan butana
5	MQ-7	sensor gas yang sensitif terhadap karbon monoksida
6	MQ-8	sensor gas yang sensitif terhadap gas hidrogen
7	MQ-9	sensor gas yang sensitif terhadap hidrogen dan gas mudah terbakar lainnya
8	MQ-131	sensor gas yang sensitif terhadap ozon
9	MQ-135	sensor gas yang sensitif terhadap benzena, alkohol, dan asap rokok
10	MQ-136	sensor gas yang sensitif terhadap gas hidrogen dan gas sulfida
11	MQ-137	sensor gas yang sensitif terhadap gas amonia
12	MQ-138	sensor gas yang sensitif terhadap benzena, toluena, alkohol, propana, formaldehida dan gas hidrogen
13	MQ-214	sensor gas yang sensitif terhadap metana dan gas alam
14	MQ-216	sensor gas yang sensitif terhadap gas alam dan gas batubara
15	MQ-303A	sensor gas yang sensitif terhadap alkohol, etanol dan asap rokok
16	MQ-306A	sensor gas yang merespon gas LPG dan butana
17	MQ-307A	sensor gas yang sensitif terhadap gas karbon monoksida
18	MQ-309A	sensor gas yang sensitif terhadap karbon monoksida dan gas mudah terbakar lainnya

2.4.2 Sensor DHT



Gambar 2. 4 Sensor DHT22

Menurut Pratama & Nugroho (2020) dalam Jurnal Teknologi dan Sistem Komputer: "Sensor DHT merupakan sensor digital yang memiliki dua parameter pengukuran, yaitu suhu dan kelembaban, yang banyak digunakan dalam sistem monitoring berbasis mikrokontroler." Sensor DHT merupakan paket sensor yang berfungsi untuk mengukur suhu dan kelembaban udara sekaligus yang dialamnya terdapat thermistor tipe NTC (*Negative Temperature Coefficient*) untuk mengukur suhu, sebuah sensor kelembapan dengan karkteristik resistif terhadap perubahan kadar air di udara serta terdapat chip yang di dalamnya melakukan beberapa konversi analog ke digital dan mengeluarkan output dengan format single-wire bi-directional (kabel tunggal dua arah)

Berikut ini adalah Spesifikasi antara DHT11 dengan DHT22 diantaranya :

Tabel 2. 5 Spesifikasi DHT11 dan DHT22

NO	DHT11	DHT22
1	Tegangan kerja = 3.3V-5V	Tegangan kerja = 3.3V-5V
2	Arus maksimum = 2.5mA	Arus maksimum = 2.5mA
3	Range pengukuran kelembaban = 20%-80%	Range pengukuran kelembaban = 0%-100%
4	Akurasi pengukuran kelembaban = 5%	Akurasi pengukuran kelembaban = 2-5%
5	Range pengukuran suhu = 0°C-50°C	Range pengukuran suhu = -40°C-80°C
6	Akurasi pengukuran suhu = 2°C	Akurasi pengukuran suhu = 0.5°C

7	Kecepatan pengambilan sampel tidak lebih dari 1 Hz (pembaruan data setiap 1 detik)	Kecepatan pengambilan sampel tidak lebih dari 0.5 Hz (pembaruan data setiap 2 detik)
8	Ukuran = 15.5 mm x 12 mm x 5.5 mm	Ukuran = 15.1 mm x 25 mm x 7.7 mm

Berdasarkan Spesifikasi kita bisa tahu bahwa DHT11 dan DHT22 ini memiliki Perbedaan yang cukup signifikan yaitu:

- Range pengukuran suhu. Dimana DHT11 (0°C-50°C), sedangkan DHT22 (-40°C-80°C).
- Akurasi pengukuran suhu. Dimana DHT11 memiliki akurasi 2°C sedangkan DHT22 adalah 0.5°C.
- Range pengukuran kelembaban. DHT11 (20%-80%) sedangkan DHT22 (0%-100%).
- Akurasi pengukuran kelembaban. DHT11 memiliki akurasi 5% sedangkan DHT22 sebesar 2%-5%.
- Kecepatan update data. DHT11 setiap 1 detik sekali sedangkan DHT22 2 detik sekali.

2.5 Kabel Jumper

Kabel Jumper merupakan kabel elektrik yang mempunyai pin konektor di setiap ujungnya dan memungkinkan untuk menghubungkan dua komponen yang melibatkan Arduino tanpa memerlukan solder. Intinya, kegunaan kabel jumper ini digunakan sebagai konduktor listrik untuk menyambungkan rangkaian listrik.

Kabel jumper biasanya digunakan pada breadboard atau alat prototyping lainnya supaya lebih mudah untuk mengutak-atik rangkaian. Konektor yang terdapat pada ujung kabel terdiri dari konektor jantan (male connector) dan konektor betina (female connector). Konektor female berfungsi untuk menusuk dan konektor male berfungsi untuk ditusuk.

Cara Kerja Kabel Jumper Sederhananya, kabel jumper bekerja untuk menghantarkan arus listrik dari satu komponen ke komponen lainnya yang

dihubungkan. Hal ini terjadi karena di ujung dan di dalam kabel terdapat konduktor listrik kecil yang memang fungsinya untuk menghantarkan listrik.

2.5.1 Jenis jenis Kabel Jumper

Kabel Jumper memiliki 3 Jenis diantaranya :

Tabel 2. 6 Jenis Kabel Jumper

No	Gambar	Jenis	Fungsi
1	 **Gambar 2. 5** Male to Male	Male to Male	Kedua ujungnya memiliki 2 Pin
2	 **Gambar 2. 6** Male to Female	Male to Female	Memiliki ujung yang berbeda 1 pin dan 1 soket
3	 **Gambar 2. 7** Female to Female	Female to Female	Kedua ujungnya memiliki 2 soket

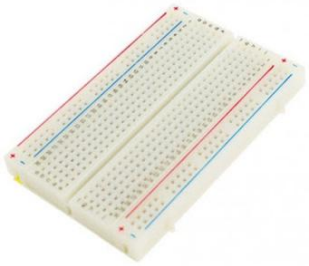
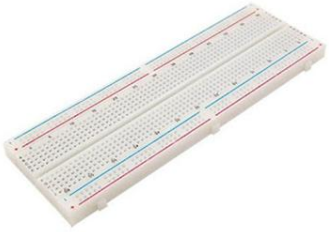
2.6 Bread Board

Menurut Putra & Susanto (2021) dalam Jurnal Teknologi dan Rekayasa: "Breadboard merupakan papan tempat merangkai komponen elektronika secara sementara yang berguna untuk percobaan dan pengujian sebelum rangkaian disolder secara permanen." Breadboard Arduino merupakan papan yang digunakan untuk membuat prototype rangkaian elektronik. Ada beberapa orang yang menyebutnya project board bahkan protoboard (*prototype board*). Pada arah vertikal masing-masing lubang saling berhubungan, namun tidak untuk arah horizontal. Breadboard berfungsi sebagai konduktor listrik sekaligus tempat melekatkan kabel jumper atau header pin male agar arus listrik dari komponen satu ke komponen lainnya bisa saling terdistribusi dengan baik sesuai keinginan tanpa harus merepotkan pengguna untuk melakukan penyolderan atau melakukan bongkar pasang.

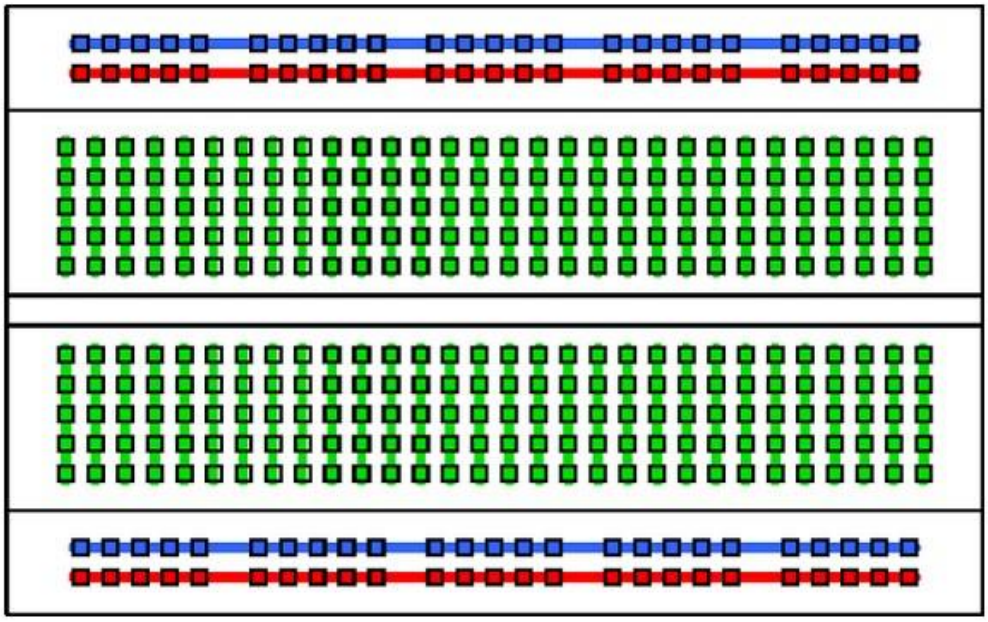
2.6.1 Jenis jenis Breadboard

Tabel 2. 7 Jenis Breadboard

NO	Gambar	Jenis	Jumlah Titik Koneksi
1	 **Gambar 2. 8** Mini Breadboard	Mini Breadboard	170 Titik Koneksi

2	 **Gambar 2. 9** Medium Breadboard	Medium Breadboard	400 Titik Koneksi
3	 **Gambar 2. 10** Large Breadboard	Large Breadboard	800 Titik Koneksi

2.6.2 Prinsip Kerja Breadboard



Gambar 2. 11 Koneksivitas antara lubang Breadboard

Untuk menghubungkan antara satu lubang dengan lubang yang lain, maka di bagian bawah lubang tersebut terdapat logam konduktor listrik yang diposisikan

secara khusus. Hal ini berguna untuk memudahkan pengguna dalam membuat rangkaian

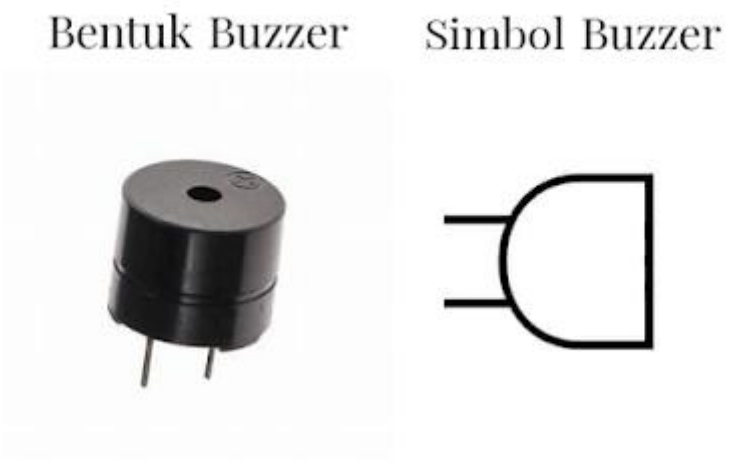
Berdasarkan gambar di atas, fungsi dari masing-masing jalur koneksi pada breadboard dengan keterangan warnanya yaitu sebagai berikut :

1. Jalur warna merah digunakan untuk menempatkan pin 5V atau kutub positif dari Arduino untuk dihubungkan ke kutub positif komponen lain.

2. Jalur warna biru digunakan untuk menempatkan pin GND atau kutub negatif dari Arduino untuk dihubungkan ke kutub negatif komponen lain.

3. Jalur warna hijau digunakan untuk menempatkan pin digital dari Arduino untuk dihubungkan ke komponen lain.

2.7. Buzzer



Gambar 2. 12 Bentuk dan Simbol dari Buzzer

2.7.1 Pengertian Buzzer Elektronika

Buzzer Elektronika adalah sebuah komponen elektronika yang dapat menghasilkan getaran suara berupa gelombang bunyi. Buzzer elektronika akan menghasilkan getaran suara ketika diberikan sejumlah tegangan listrik dengan taraf tertentu sesuai dengan spesifikasi bentuk dan ukuran buzzer elektronika itu sendiri.

Pada umumnya, buzzer elektronika ini sering digunakan sebagai alarm karena penggunaannya yang cukup mudah yaitu dengan memberikan tegangan input maka buzzer elektronika akan menghasilkan getaran suara berupa gelombang bunyi yang dapat didengar manusia.

Pada dasarnya, setiap buzzer elektronika memerlukan input berupa tegangan listrik yang kemudian diubah menjadi getaran suara atau gelombang bunyi yang memiliki frekuensi berkisar antara 1 - 5 KHz. Jenis buzzer elektronika yang sering digunakan dan ditemukan dalam rangkaian adalah buzzer yang berjenis Piezoelectric (Piezoelectric Buzzer). Hal itu karena *Piezoelectric Buzzer* memiliki berbagai kelebihan diantaranya yaitu lebih murah, relatif lebih ringan dan lebih mudah penggunaannya ketika diaplikasikan dalam rangkaian elektronika.

Efek Piezoelektrik (*Piezoelectric Effect*) ditemukan pertama kali oleh dua orang ilmuwan Fisika pada tahun 1880 bernama Pierre Curie dan Jacques Curie yang berasal dari kebangsaan Perancis. Penemuan tersebut kemudian dikembangkan oleh sebuah perusahaan Jepang menjadi *Piezoelectric Buzzer* dan mulai populer digunakan pada tahun 1970-an.

2.7.2 Prinsip Kerja Buzzer Elektronika

Pada dasarnya, prinsip kerja dari buzzer elektronika hampir sama dengan loud speaker dimana buzzer juga terdiri dari kumparan yang terpasang secara diafragma. Ketika kumparan tersebut dialiri listrik maka akan menjadi elektromagnet sehingga mengakibatkan kumparan tertarik ke dalam ataupun ke luar tergantung dari arah arus dan polaritas magnetnya. Karena kumparan dipasang secara diafragma maka setiap kumparan akan menggerakkan diafragma tersebut secara bolak-balik sehingga membuat udara bergetar yang akan menghasilkan suara.

Namun dibandingkan dengan loud speaker, buzzer elektronika relatif lebih mudah untuk digerakkan. Sebagai contoh, buzzer elektronika dapat langsung diberikan tegangan listrik dengan taraf tertentu untuk dapat menghasilkan suara. Hal ini tentu berbeda dengan loud speaker yang memerlukan rangkaian penguat

khusus untuk menggerakkan speaker agar menghasilkan suara yang dapat didengar oleh manusia.

2.8 Arduino IDE

Arduino IDE (*Integrated Development Environment*) adalah perangkat lunak yang digunakan untuk memprogram dan mengunggah kode ke papan Arduino. Ini adalah alat inti yang digunakan oleh para pengembang, pemula, dan pecinta elektronika untuk mengembangkan berbagai macam proyek berbasis Arduino.

Beberapa fitur utama dari Arduino IDE meliputi :

- Pemrograman Bahasa C/C++: Arduino IDE memungkinkan Anda untuk menulis kode menggunakan bahasa pemrograman C/C++ yang dapat dijalankan pada mikrokontroler Arduino.
- Pustaka dan Contoh Kode: IDE menyertakan berbagai pustaka dan contoh kode yang memudahkan Anda dalam mengembangkan proyek. Pustaka-pustaka ini mempermudah akses ke fungsi dan perangkat keras tertentu.
- Editor Kode: Arduino IDE menyediakan editor kode yang sederhana dan intuitif dengan penyorotan sintaks untuk membantu Anda menulis kode dengan lebih mudah.
- Compiler dan Uploader: Setelah menulis kode, Anda dapat mengompilasi program Anda dengan mudah menggunakan Arduino IDE. Kemudian, IDE juga memungkinkan Anda untuk mengunggah program yang telah dikompilasi ke papan Arduino melalui koneksi USB.
- Monitor Serial: Arduino IDE memiliki fitur Monitor Serial yang memungkinkan Anda untuk memantau dan berkomunikasi dengan perangkat Arduino melalui koneksi serial.
- Dukungan untuk Berbagai Board: Arduino IDE mendukung berbagai jenis papan Arduino dan papan mikrokontroler yang kompatibel dengan bahasa pemrograman Arduino.

- Pemecahan Masalah: Jika terjadi kesalahan dalam kode Anda, Arduino IDE akan memberikan pesan kesalahan dan membantu Anda untuk memperbaikinya.

2.9 Blynk



Gambar 2. 13 Logo Blynk

Blynk merupakan salah satu platform *Internet of Things* (IoT) yang menyediakan infrastruktur cloud dan aplikasi antarmuka pengguna, dirancang untuk memudahkan interaksi antara perangkat keras elektronik dengan pengguna akhir melalui koneksi internet. Platform ini menjembatani kesenjangan antara dunia fisik perangkat IoT dan dunia digital antarmuka pengguna, memungkinkan pengembang maupun non-pengembang untuk menciptakan proyek IoT yang terkoneksi dengan cepat dan intuitif.

Inti dari Blynk terletak pada kemampuannya untuk menyediakan jembatan komunikasi yang mulus antara perangkat keras mikrokontroler, seperti ESP32 yang digunakan dalam sistem ini, dengan server berbasis cloud. Perangkat keras mengirimkan dan menerima data melalui jaringan internet (dalam kasus ini, Wi-Fi) ke dan dari server Blynk. Server ini kemudian berfungsi sebagai pusat penghubung, menyimpan data, mengelola logika proyek, dan meneruskan informasi ke aplikasi seluler atau dashboard web pengguna.

Di sisi pengguna, Blynk menawarkan aplikasi seluler dan dashboard web yang sangat fleksibel, memungkinkan pembuatan antarmuka grafis yang sepenuhnya

dapat disesuaikan. Pengguna dapat memilih dari berbagai macam widget siap pakai, seperti pengukur (*gauge*), tampilan nilai (*value display*), grafik riwayat (*history graph*), tombol, dan sakelar, yang dapat diatur sesuai kebutuhan proyek. Widget-widget ini dihubungkan ke "pin virtual" pada perangkat keras, sebuah konsep abstrak yang membebaskan pengembang dari keterikatan pada pin fisik mikrokontroler. Melalui pin virtual ini, data sensor yang dikumpulkan dari perangkat keras dapat divisualisasikan secara real-time di antarmuka pengguna, dan sebaliknya, perintah dari pengguna dapat dikirimkan kembali ke perangkat keras untuk mengontrol aktuator atau mengubah parameter.

Selain visualisasi dan kontrol, Blynk juga menyediakan fitur notifikasi, memungkinkan sistem untuk mengirimkan peringatan atau informasi penting kepada pengguna melalui push notification, email, atau SMS, berdasarkan kondisi yang diprogram dalam perangkat keras atau di server. Dengan demikian, Blynk menawarkan ekosistem yang komprehensif untuk pengembangan IoT, menyederhanakan proses konektivitas, visualisasi data, dan interaksi pengguna, sehingga fokus dapat lebih dicurahkan pada fungsionalitas inti proyek itu sendiri..

2.10 Diagram Blok

Diagram blok adalah representasi grafis dari suatu sistem, proyek, atau skenario. Diagram ini memberikan gambaran fungsional suatu sistem dan menggambarkan bagaimana berbagai elemen dalam sistem tersebut saling terkait. Dalam representasi ini, elemen-elemen utama dalam sistem disajikan dalam bentuk blok-blok yang saling terhubung melalui garis atau panah, yang menunjukkan aliran data, sinyal, atau informasi di antara masing-masing komponen.

Blok diagram memiliki peranan penting di berbagai bidang, seperti teknik, ilmu komputer, dan penelitian ilmiah, karena mampu menyederhanakan sistem yang kompleks menjadi bagian-bagian fungsional yang lebih mudah dianalisis. Setiap blok dalam diagram biasanya merepresentasikan suatu proses, fungsi, atau komponen tertentu, sedangkan garis penghubung menunjukkan interaksi atau hubungan fungsional antar bagian sistem. Melalui pendekatan ini, blok diagram menjadi alat bantu yang efektif dalam memahami mekanisme kerja sistem secara

menyeluruh serta interaksi antar subsistem yang ada di dalamnya. Komponen Utama dalam Blok Diagram yaitu,

- Blok : Mewakili komponen atau fungsi utama dalam sistem.
- Garis atau Panah : Menunjukkan aliran informasi atau sinyal antar blok.
- Label : Memberikan informasi tambahan tentang fungsi atau nama komponen dalam blok

2.11 Flowchart

Menurut Agustinus Zalukhu, Swingly Purba, Dedi Darma (2023) Flowchart adalah penggambaran secara grafik dari langkah-langkah dan urutan urutan prosedur dari suatu program. Flowchart atau bagan alur adalah diagram yang menampilkan langkah-langkah dan keputusan untuk melakukan sebuah proses dari suatu program. Setiap langkah digambarkan dalam bentuk diagram dan dihubungkan dengan garis atau arah panah.

Flowchart berperan penting dalam memutuskan sebuah langkah atau fungsionalitas dari sebuah proyek pembuatan program yang melibatkan banyak orang sekaligus. Selain itu dengan menggunakan bagan alur proses dari sebuah program akan lebih jelas, ringkas, dan mengurangi kemungkinan untuk salah penafsiran. Penggunaan flowchart dalam dunia pemrograman juga merupakan cara yang bagus untuk menghubungkan antara kebutuhan teknis dan non-teknis

Fungsi utama dari flowchart adalah memberi gambaran jalannya sebuah program dari satu proses ke proses lainnya. Sehingga, alur program menjadi mudah dipahami oleh semua orang. Selain itu, fungsi lain dari flowchart adalah untuk menyederhanakan rangkaian prosedur agar memudahkan pemahaman terhadap informasi tersebut

2.11.1 Jenis Flowchart

- Flowchart dokumen

Pertama ada flowchart dokumen (document flowchart) atau bisa juga disebut dengan paperwork flowchart. Flowchart dokumen berfungsi untuk menelusuri alur form dari satu bagian ke bagian yang lain, termasuk bagaimana laporan diproses, dicatat, dan disimpan.

- Flowchart program

Selanjutnya kita akan membahas flowchart program. Flowchart ini menggambarkan secara rinci prosedur dari proses program. Flowchart program terdiri dari dua macam, antara lain: flowchart logika program (program logic flowchart) dan flowchart program komputer terinci (*detailed computer program flowchart)*.

- Flowchart sistem

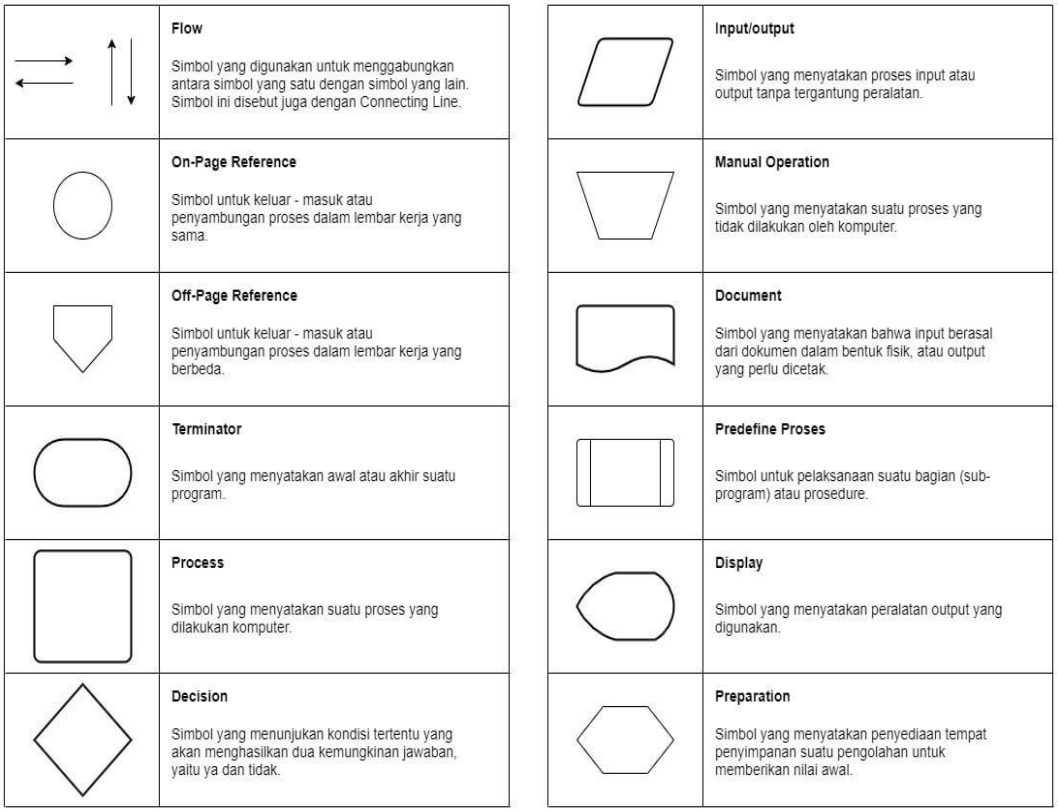
Yang keempat ada flowchart sistem. Flowchart sistem adalah flowchart yang menampilkan tahapan atau proses kerja yang sedang berlangsung di dalam sistem secara menyeluruh. Selain itu flowchart sistem juga menguraikan urutan dari setiap prosedur yang ada di dalam sistem.

- Flowchart skematik

Terakhir ada flowchart skematik. Flowchart ini menampilkan alur prosedur suatu sistem, hampir sama dengan flowchart sistem. Namun, ada perbedaan dalam penggunaan simbol-simbol dalam menggambarkan alur. Selain simbol-simbol, flowchart skematik juga menggunakan gambar-gambar komputer serta peralatan lainnya untuk mempermudah dalam pembacaan flowchart untuk orang awam.

2.11.2 Simbol pada flowchart

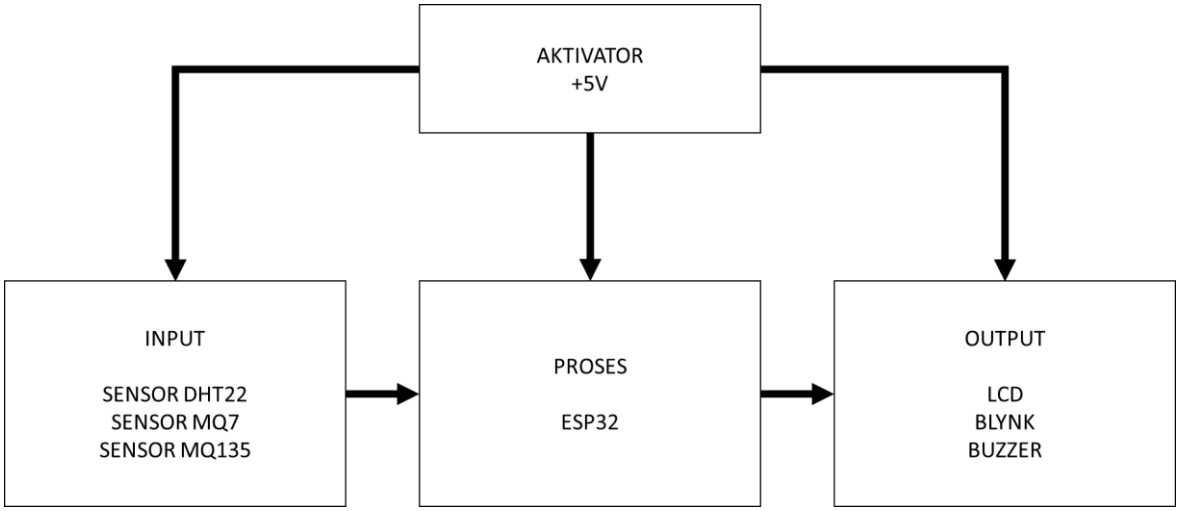
Pada dasarnya simbol-simbol dalam flowchart memiliki arti yang berbeda-beda. Berikut adalah simbol-simbol yang sering digunakan dalam proses pembuatan flowchart.



Gambar 2. 14 Simbol simbol pada Flowchart

3. PEMBAHASAN

3.1. Blok Diagram



Gambar 3. 1 Blok Diagram

Alat ini menggunakan activator dengan tegangan 5 volt dan 3.3 volt. Input pada alat berupa sensor DHT22, MQ7, dan sensor MQ135. Untuk mikrokontroller yang digunakan adalah ESP32 dikarenakan ESP32 sudah mendukung Modul WiFi dan Bluetooth. Output yang dihasilkan adalah LCD dan Monitoring dari blynk secara Realtime dan Buzzer Ketika polutan melebihi ambang batas

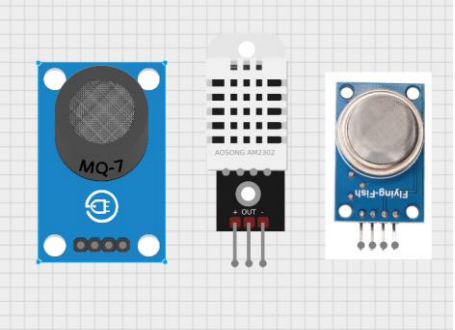
3.1.1 Blok Aktivator



Gambar 3. 2 Blok Aktivator

Blok aktivator yang digunakan pada prototipe monitoring kualitas udara berbasis ESP32 bekerja dengan tegangan +5V dan +3.3V sebagai sumber daya utama untuk menjalankan Sistem. Daya +5V dan +3.3V ini didapatkan Melalui Power Supply Eksternal yang kemudian didistribusikan untuk beban sistem

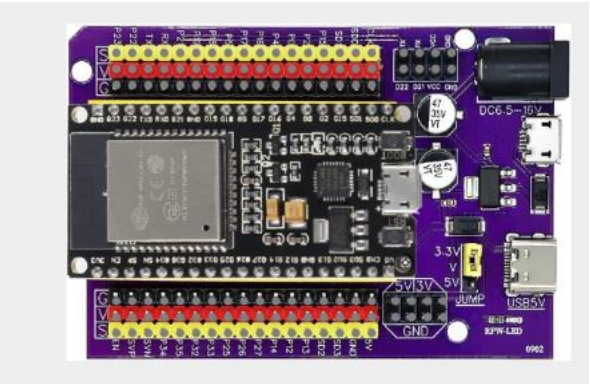
3.1.2 Blok Input



Gambar 3. 3 Blok Input

Blok input pada Prototipe Sistem Monitoring Udara berbasis ESP32 memiliki peran penting untuk mengumpulkan data lingkungan secara akurat, menjadi fondasi utama bagi Prototipe Sistem kualitas udara. Bagian ini diwujudkan melalui Fungsi ketiga komponen sensor sensor diantaranya MQ-7 yang secara spesifik mendeteksi konsentrasi Karbon Monoksida (CO) dan menghasilkan sinyal tegangan analog, sensor MQ-135 yang bertanggung jawab atas pemantauan polutan udara dengan keluaran analog, serta sensor DHT22 yang bertugas mengukur kondisi suhu dan kelembaban relatif dengan output digital. Ketiga sensor ini secara kolektif memastikan mikrokontroler ESP32 menerima data yang lengkap, mulai dari keberadaan gas-gas berbahaya hingga kondisi Udara yang relevan, yang kemudian diolah sebagai dasar untuk analisis kualitas udara.

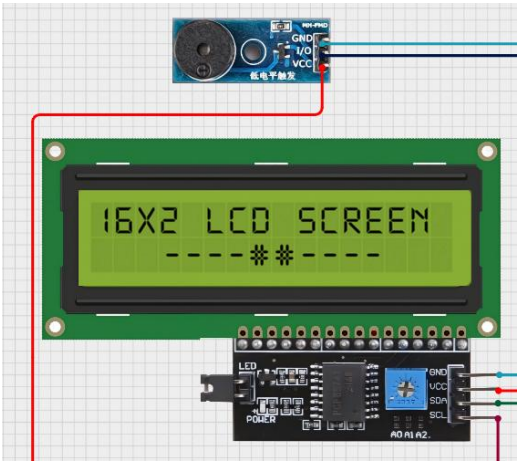
3.1.3 Blok Proses



Gambar 3. 4 ESP32 yang Terhubung dengan Baseboard

Blok proses dalam Prototipe Sistem Monitoring Udara ini menggunakan ESP32, sebuah mikrokontroler berdaya rendah dengan kapabilitas Wi-Fi dan Bluetooth terintegrasi. Sebagai otak dari keseluruhan sistem, ESP32 memiliki fungsi penting dalam menerima, mengolah, dan membuat keputusan berdasarkan data yang dikumpulkan oleh blok input. Setelah mendapatkan sinyal analog dari sensor MQ-7 dan MQ-135, ESP32 akan melakukan konversi analog-ke-digital, kemudian mengaplikasikan algoritma matematis yang kompleks, melibatkan nilai resistansi referensi (Ro) dan konstanta spesifik sensor (a dan b) yang diambil dari datasheet, untuk mentransformasi data mentah tersebut menjadi kadar polutan dalam satuan Parts Per Million (PPM) yang berarti (yaitu Karbon Monoksida dari MQ-7 dan polutan umum dari MQ-135). Bersamaan dengan itu, ESP32 juga berkomunikasi secara digital dengan sensor DHT22 untuk memperoleh data suhu dan kelembaban udara. Setelah semua data sensor dikonversi dan dihitung ke dalam satuan yang relevan, ESP32 akan mengevaluasinya terhadap ambang batas kualitas udara yang telah ditentukan, dan berdasarkan perbandingan ini akan menetapkan status kualitas udara sebagai "Baik" atau "Buruk". Selain pemrosesan data sensor, ESP32 juga mengelola seluruh komunikasi nirkabel, memanfaatkan modul Wi-Fi internalnya untuk mengirimkan data yang sudah terproses ini ke platform IoT Blynk secara real-time, serta memberikan data untuk ditampilkan pada LCD dan Ketika Kualitas udara melebihi ambang batas maka Buzzer akan berbunyi yang menandakan kualitas udara “Buruk” dan akan menampilkan pada LCD dan Blynk.

3.1.4 Blok Output

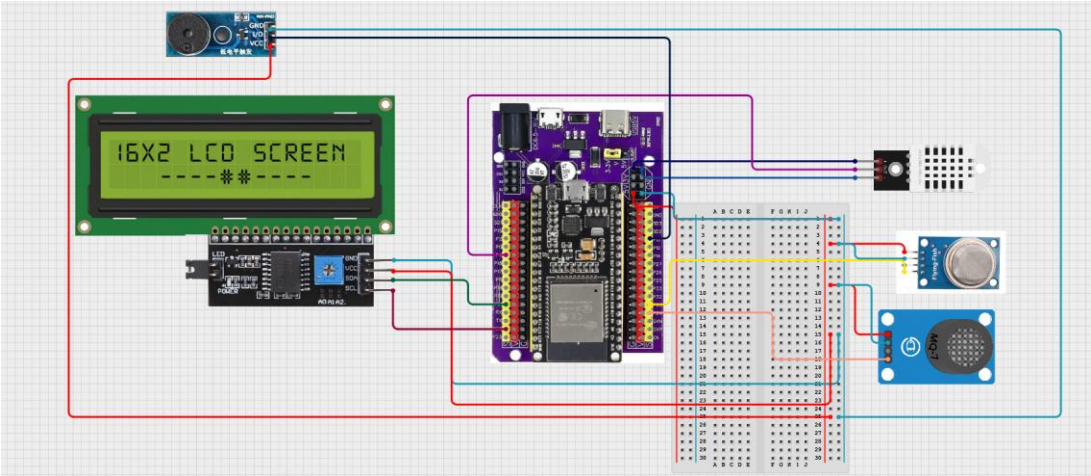


Gambar 3. 5 Rangkaian pada Blok Output

Blok output pada prototipe sistem ini dirancang untuk menyajikan data kualitas udara yang telah diproses oleh ESP32 kepada pengguna melalui dua media utama diantaranya tampilan lokal pada *Liquid Crystal Display* (LCD) dan visualisasi jarak jauh melalui platform *Internet of Things* (IoT) Blynk. LCD I2C berfungsi sebagai antarmuka output langsung yang terpasang pada perangkat, menampilkan informasi esensial secara real-time kepada pengguna yang berada di dekat sistem. Layar ini secara efisien menampilkan suhu dan kelembaban udara yang terukur dari sensor DHT22, serta status kualitas udara yang telah dianalisis (baik atau buruk), memastikan akses cepat terhadap informasi krusial tanpa memerlukan koneksi internet. Sementara itu, Blynk melengkapi fungsionalitas output dengan menyediakan platform visualisasi data berbasis cloud yang komprehensif, memungkinkan pengguna untuk memantau seluruh parameter kualitas udara dari mana saja melalui aplikasi smartphone atau dashboard web. Di Blynk, data disajikan dalam berbagai format widget interaktif, seperti gauge untuk menampilkan nilai numerik suhu, kelembaban, konsentrasi Karbon Monoksida (CO) dalam PPM, dan polutan udara umum dalam PPM, serta sebuah label khusus untuk status kualitas udara (baik/buruk). Kombinasi kedua metode output ini memastikan pengguna mendapatkan informasi kualitas udara yang akurat dan mudah diakses, baik secara lokal maupun dari jarak jauh, sesuai dengan kebutuhan

pemantauan yang fleksibel serta Penggunaan Buzzer sebagai Pemberitahuan secara Real time Ketika kualitas udara melebihi ambang batas dan bisa di mati/nyalakan melalui Blynk.

3.2 Analisa Rangkaian



Gambar 3. 6 Rangkaian Keseluruhan

Analisis rangkaian ini menjelaskan hubungan antar komponen utama dengan aliran sinyal pada prototipe sistem monitoring kualitas udara. Mikrokontroler ESP32 berfungsi sebagai unit pemroses sentral yang mengelola data, pemrosesan, dan distribusi data.

Sistem ini memperoleh catu daya utama dari sumber 5V melalui port USB pada ESP32. Tegangan 5V ini selanjutnya digunakan untuk menyuplai daya ke modul sensor MQ-7 dan MQ-135, serta modul LCD I2C. ESP32 sendiri beroperasi pada 3.3V yang diregulasi secara internal. Semua komponen berbagi koneksi ground (GND) yang umum untuk memastikan stabilitas sirkuit. Sensor DHT22 ditenagai langsung dari pin 3.3V ESP32.

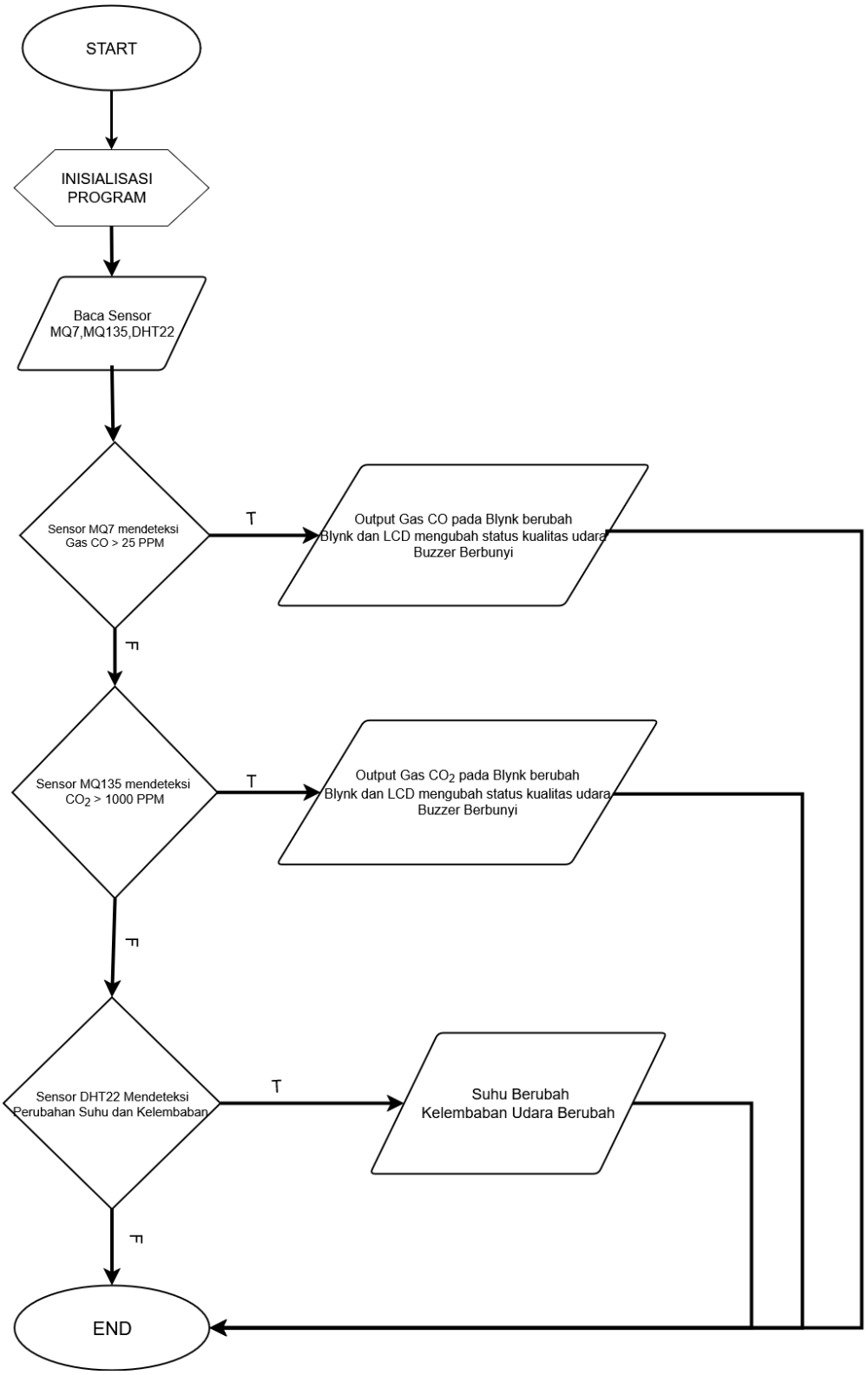
Aliran sinyal input dimulai dari sensor MQ-7 dan MQ-135, yang menghasilkan sinyal tegangan analog. Sinyal ini disalurkan ke pin ADC (Analog-to-Digital Converter) pada ESP32, yaitu GPIO34 untuk MQ-7 dan GPIO35 untuk MQ-135. ESP32 mengkonversi tegangan analog ini menjadi nilai digital. Sensor DHT22, sebagai input digital, terhubung ke pin GPIO4 dan berkomunikasi dengan ESP32

menggunakan protokol data serial satu kabel untuk mengirimkan data suhu dan kelembaban.

Data yang dikumpulkan kemudian diproses oleh ESP32. Nilai digital dari sensor MQ diolah menggunakan perhitungan matematis yang melibatkan resistansi sensor (Rs), resistansi referensi udara bersih (Ro), dan konstanta spesifik sensor (a, b) untuk menghasilkan konsentrasi gas dalam satuan *Parts Per Million* (PPM). Data suhu dan kelembaban dari DHT22 juga diintegrasikan. Berdasarkan nilai PPM yang dihitung dan ambang batas yang ditetapkan, ESP32 menentukan status kualitas udara secara logis sebagai "Baik" atau "Buruk".

Untuk output, informasi yang telah diproses disajikan melalui dua kanal. LCD I2C terhubung melalui jalur data serial SDA (GPIO21) dan SCL (GPIO22), menerima data dari ESP32 untuk ditampilkan secara lokal (suhu, kelembaban, dan status kualitas udara). Secara bersamaan, ESP32 memanfaatkan modul Wi-Fi internalnya untuk mengirimkan seluruh data sensor yang telah diolah, termasuk nilai PPM, ke platform IoT Blynk. Komunikasi ini memungkinkan visualisasi data secara real-time dan pemantauan jarak jauh melalui aplikasi seluler atau dashboard web dengan Buzzer yang digunakan sebagai Alat pemberitahu Ketika polutan melebihi ambang batas.

3.3 Analisa Flowchart



Gambar 3. 7 Flowchart Prototipe Monitoring Udara Berbasis ESP32

Flowchart yang ditampilkan menggambarkan alur kerja sistem monitoring udara secara berurutan, dimulai dengan inisialisasi program. Setelah inisialisasi, sistem secara berurutan membaca data dari sensor MQ-7, MQ-135, dan DHT22. Setiap pembacaan sensor diikuti oleh blok keputusan yang memeriksa apakah terjadi "perubahan" pada data yang terdeteksi.

Menurut Yeni Irdayanti, Johansyah Alrasyid, Ahlika Azzahra (2025) ambang batas yang ditetapkan untuk menentukan kualitas udara sebagai "Buruk" adalah untuk Gas Karbon Monoksida (CO) dari sensor MQ-7, ambang batasnya Adalah 25.0 PPM, sementara untuk Karbon Dioksida (CO_2) dari sensor MQ-135, ambang batasnya adalah 1000.0 PPM. Jika sensor MQ-7 mendeteksi perubahan konsentrasi Karbon Monoksida (CO), atau sensor MQ-135 mendeteksi perubahan konsentrasi Karbon Dioksida (CO2), maka output gas pada Blynk akan berubah dan status udara akan diperbarui. Demikian pula, jika sensor DHT22 mendeteksi perubahan suhu atau kelembaban, informasi tersebut akan diperbarui.

Secara konseptual, flowchart ini menekankan bahwa pembaruan output hanya dipicu oleh adanya perubahan pada data sensor. Namun, perlu dicatat bahwa alur ini menuju END setelah satu siklus pembacaan dan pembaruan, yang tidak menggambarkan karakteristik sebuah sistem monitoring yang dirancang untuk beroperasi secara kontinu. Untuk pemantauan real-time yang berkelanjutan, alur program seharusnya kembali ke fase pembacaan sensor setelah menyelesaikan semua proses output, membentuk sebuah loop tak berujung.

3.4 Analisa Program

3.4.1 Definisi Awal dan Inisialisasi Pustaka

Tabel 3. 1 Definisi Awal dan Inisialisasi Pustaka

Kode Program	Penjelasan
#define BLYNK_TEMPLATE_ID "Isi dengan Template Blynk" #define BLYNK_TEMPLATE_NAME "Monitoring Kualitas Udara" #define BLYNK_PRINT Serial	• #define BLYNK_TEMPLATE_ID dan #define BLYNK_TEMPLATE_NAME: Mendefinisikan ID dan nama template Blynk untuk proyek ini. Ini penting agar perangkat ESP32

#include <WiFi.h> #include <WiFiClient.h> #include <BlynkSimpleEsp32.h> #include <DHT.h> #include <LiquidCrystal_I2C.h>	Anda terhubung ke dashboard Blynk yang benar. • #define BLYNK_PRINT Serial: Mengaktifkan *debugging output* dari Blynk ke Serial Monitor, sangat membantu untuk melihat status koneksi dan komunikasi. • #include <WiFi.h>: Pustaka untuk mengelola koneksi Wi-Fi. • #include <WiFiClient.h>: Pustaka untuk klien Wi-Fi yang diperlukan oleh Blynk. • #include <BlynkSimpleEsp32.h>: Pustaka utama Blynk untuk ESP32. • #include <DHT.h>: Pustaka untuk berkomunikasi dengan sensor suhu dan kelembaban DHT (misalnya DHT22). • #include <LiquidCrystal_I2C.h>: Pustaka untuk mengontrol modul LCD yang terhubung melalui antarmuka I2C.

3.4.2 Kredensial dan Konfigurasi Hardware

Tabel 3. 2 Kredensial dan Konfigurasi Hardware

Kode Program	Penjelasan
char ssid[] = "Isi dengan SSID"; char pass[] = "Isi dengan Password WIfi"; char auth[] = "Isi dengan Autentikasi Blynk"; LiquidCrystal_I2C lcd(0x27, 16, 2); #define DHTPIN 4 #define DHTTYPE DHT22 DHT dht(DHTPIN, DHTTYPE); #define MQ7_PIN 34 #define MQ135_PIN 35 #define BUZZER_PIN 13	• char ssid[], char pass[]: Menyimpan kredensial jaringan Wi-Fi (nama Wi-Fi dan kata sandi) yang akan digunakan ESP32 untuk terhubung ke internet. Anda harus menggantinya dengan kredensial Wi-Fi Anda sendiri. • char auth[]: Menyimpan Auth Token Blynk yang unik untuk proyek Anda. Token ini didapatkan dari email Blynk setelah membuat proyek baru. Anda harus menggantinya dengan Auth Token Anda. • LiquidCrystal_I2C lcd(0x27, 16, 2): Membuat

	objek lcd untuk mengontrol LCD I2C. • 0x27 adalah alamat I2C default untuk banyak modul LCD I2C. Pastikan alamat ini sesuai dengan modul Anda (bisa berbeda, misal 0x3F). • 16, 2 menunjukkan ukuran LCD: 16 kolom dan 2 baris. • #define DHTPIN 4: Menentukan bahwa pin data sensor DHT22 terhubung ke GPIO 4 pada ESP32. • #define DHTTYPE DHT22: Menentukan jenis sensor DHT yang digunakan adalah DHT22. • DHT dht(DHTPIN, DHTTYPE): Membuat objek dht untuk berkomunikasi dengan sensor DHT. • #define MQ7_PIN 34: Menentukan pin analog ESP32 (GPIO34) yang terhubung ke output analog sensor MQ-7 (Karbon Monoksida). • #define MQ135_PIN 35: Menentukan pin analog ESP32 (GPIO35) yang terhubung ke output analog sensor MQ-135 (polutan udara). • #define BUZZER_PIN 13: Menentukan pin digital ESP32 (GPIO13) yang terhubung ke buzzer. Ini diasumsikan sebagai buzzer aktif, yang hanya perlu diberikan tegangan HIGH untuk berbunyi.

3.4.3 Konstanta dan Variabel Global

Tabel 3. 3 Konstanta dan Variabel Global

Kode Program	Penjelasan

const float VREF = 3.3;
const float ADC_MAX = 4095.0;
 	
const float RL_MQ7 = 10.0;
const float MQ7_CO_A = 9.87;
const float MQ7_CO_B = -0.925;
float Ro_MQ7 = 0.0;
 	
const float RL_MQ135 = 20.0;
const float MQ135_AIR_A = 116.6020682;
const float MQ135_AIR_B = -2.769034857;
float Ro_MQ135 = 0.0;
 	
const float CO_PPM_THRESHOLD_BAD = 25.0;
const float AIR_QUALITY_PPM_THRESHOLD_BAD = 1000.0;
 	
float temperature;
float humidity;
float co_ppm;
float air_quality_ppm;
String airQualityStatus = "Unknown";
 	
BlynkTimer timer;
 	
bool buzzerActive = false;
bool buzzerManualOverrideState = false;
	• const float VREF = 3.3;: Tegangan referensi analog-ke-digital converter (ADC) ESP32 (biasanya 3.3V).
	
	• const float ADC_MAX = 4095.0;: Nilai maksimum pembacaan ADC untuk ESP32 (12-bit ADC, sehingga 212−1=4095).
	
	• RL_MQ7, MQ7_CO_A, MQ7_CO_B, Ro_MQ7: Konstanta dan variabel untuk perhitungan konsentrasi Karbon Monoksida (CO) dari sensor MQ-7.
	
	• RL adalah resistansi beban (Load Resistor) yang digunakan bersama sensor MQ.
	
	  • A dan B adalah koefisien dari kurva sensitivitas sensor yang digunakan untuk mengkonversi rasio Rs/Ro ke PPM (bagian per juta).
	
	  • Ro adalah resistansi sensor di udara bersih, yang akan dikalibrasi.
	
	  • RL_MQ135, MQ135_AIR_A, MQ135_AIR_B, Ro_MQ135: Konstanta dan variabel yang serupa untuk perhitungan kualitas udara dari sensor MQ-135.
	
	• CO_PPM_THRESHOLD_BAD: Ambang batas PPM CO. Jika konsentrasi CO melebihi nilai ini, kualitas udara dianggap buruk.
	
	• AIR_QUALITY_PPM_THRESHOLD_BAD: Ambang batas PPM polutan udara umum. Jika konsentrasi polutan melebihi nilai ini, kualitas udara dianggap buruk.

	• temperature, humidity, co_ppm, air_quality_ppm, airQualityStatus: Variabel untuk menyimpan nilai-nilai sensor yang dibaca dan status kualitas udara. • BlynkTimer timer;: Membuat objek timer Blynk untuk menjalankan fungsi-fungsi tertentu secara periodik tanpa memblokir loop(). • bool buzzerActive = false;: Variabel boolean yang melacak status fisik buzzer (apakah sedang berbunyi atau tidak). • bool buzzerManualOverrideState = false;: Variabel boolean yang melacak apakah pengguna telah secara manual mengaktifkan atau menonaktifkan buzzer dari Blynk.

3.4.4 Fungsi Pembantu Perhitungan Sensor MQ

Tabel 3. 4 Fungsi Pembantu Perhitungan Sensor MQ

Kode Pemograman	Penjelasan

float calculateRs(int adcValue, float R_L) {
  float Vout = adcValue * (VREF / ADC_MAX);
  float Rs = (VREF / Vout - 1) * R_L;
  return Rs;
}
 	
float calculatePPM(float rs_ro_ratio, float a_const, float b_const) {
  if (rs_ro_ratio <= 0) return 0;
  return a_const * pow(rs_ro_ratio, b_const);
}
	• calculateRs(int adcValue, float R_L): Fungsi ini menghitung resistansi sensor (Rs) berdasarkan nilai ADC yang dibaca dan resistansi beban (R_L).
	  • Pertama, nilai ADC dikonversi menjadi tegangan Vout.
	  • Kemudian, Rs dihitung menggunakan rumus pembagi tegangan.
	• calculatePPM(float rs_ro_ratio, float a_const, float b_const): Fungsi ini menghitung konsentrasi gas dalam PPM (Parts Per Million) dari rasio Rs/Ro (rs_ro_ratio)

	menggunakan rumus daya yang didapat dari datasheet sensor (atau grafik sensitivitas). a_const dan b_const adalah koefisien spesifik untuk setiap gas dan sensor.

3.4.5 Fungsi Kalibrasi Sensor MQ

Tabel 3. 5 Fungsi Kalibrasi Sensor MQ

Kode Program	Penjelasan

void calibrateSensors() {
  Serial.println("\nMemulai kalibrasi sensor MQ... Pastikan sensor berada di udara bersih dan stabil.");
  Serial.println("Ini akan memakan waktu sekitar 10 detik...");
  lcd.clear();
  lcd.print("Kalibrasi MQ...");
  lcd.setCursor(0,1);
  lcd.print("Udara Bersih!");
  delay(10000);
 	
  float RsSum_MQ7 = 0;
  float RsSum_MQ135 = 0;
  int numReadings = 50;
 	
  for (int i = 0; i < numReadings; i++) {
  RsSum_MQ7 += calculateRs(analogRead(MQ7_PIN), RL_MQ7);
  RsSum_MQ135 += calculateRs(analogRead(MQ135_PIN), RL_MQ135);
  delay(100);
  }
 	
  Ro_MQ7 = RsSum_MQ7 / numReadings;
  Ro_MQ135 = RsSum_MQ135 / numReadings;
 	
  Serial.print("Kalibrasi selesai.");
	• calibrateSensors(): Fungsi ini melakukan kalibrasi awal untuk menentukan nilai Ro (resistansi sensor di udara bersih) untuk kedua sensor MQ.
	
	• Penting untuk menjalankan kalibrasi ini di lingkungan yang bersih untuk mendapatkan nilai referensi yang akurat.
	
	• ESP32 akan menunggu 10 detik dan kemudian mengambil 50 pembacaan dari setiap sensor, merata-ratakannya untuk mendapatkan Ro_MQ7 dan Ro_MQ135.
	
	• Nilai Ro ini sangat penting untuk perhitungan PPM yang akurat.
	
	• Informasi kalibrasi juga ditampilkan di Serial Monitor dan LCD.

Serial.print(" Ro MQ-7 (CO) = ");
Serial.print(Ro_MQ7, 2);
Serial.println(" kOhm");
Serial.print(" Ro MQ-135 (Air) = ");
Serial.print(Ro_MQ135, 2);
Serial.println(" kOhm");
 	
lcd.clear();
lcd.print("Kalibrasi OK!");
lcd.setCursor(0,1);
lcd.print("Ro7:");
lcd.print(Ro_MQ7,0); lcd.print("
Ro135:"); lcd.print(Ro_MQ135,0);
delay(2000);
}	

3.4.6 Fungsi Kontrol Buzzer dari Blynk

Tabel 3. 6 Fungsi Kontrol Buzzer dari Blynk

Kode Program	Penjelasan

BLYNK_WRITE(V5) {
int pinValue = param.asInt();
 	
if (pinValue == 1) {
buzzerManualOverrideState = true;
Serial.println("Buzzer MANUAL ON dari Blynk.");
} else {
buzzerManualOverrideState = false;
Serial.println("Buzzer MANUAL OFF dari Blynk.");
}
}
	• BLYNK_WRITE(V5): Ini adalah callback function Blynk. Fungsi ini akan secara otomatis dipanggil setiap kali nilai widget (misalnya, widget Switch) yang terhubung ke Virtual Pin V5 di aplikasi Blynk berubah.
	
	• int pinValue = param.asInt();: Mengambil nilai integer dari widget (biasanya 0 untuk OFF, 1 untuk ON).
	
	• Logika if-else ini mengatur variabel buzzerManualOverrideState berdasarkan input dari Blynk. Ini memungkinkan pengguna untuk secara manual mengontrol buzzer, menimpa kontrol otomatis berdasarkan kondisi udara.

3.4.7 Fungsi Pembacaan dan Pemrosesan Data Sensor

Tabel 3. 7 Fungsi Pembacaan dan Pemrosesan Data Sensor

Kode Program	Penjelasan

void readAndProcessSensorData() {
  temperature = dht.readTemperature();
  humidity = dht.readHumidity();
 	
  float rs_mq7_current = calculateRs(analogRead(MQ7_PIN), RL_MQ7);
  float rs_ro_mq7 = rs_mq7_current / Ro_MQ7;
  co_ppm = calculatePPM(rs_ro_mq7, MQ7_CO_A, MQ7_CO_B);
 	
  float rs_mq135_current = calculateRs(analogRead(MQ135_PIN), RL_MQ135);
  float rs_ro_mq135 = rs_mq135_current / Ro_MQ135;
  air_quality_ppm = calculatePPM(rs_ro_mq135, MQ135_AIR_A, MQ135_AIR_B);
 	
  bool buzzerAutoShouldBeOn = false;
  if (co_ppm > CO_PPM_THRESHOLD_BAD || air_quality_ppm > AIR_QUALITY_PPM_THRESHOLD_BAD) {
  buzzerAutoShouldBeOn = true;
  }
 	
  if (isnan(temperature) || isnan(humidity)) {
  airQualityStatus = "DHT Error";
  } else {
  if (buzzerAutoShouldBeOn) {
  airQualityStatus = "Buruk";
  } else {
  airQualityStatus = "Baik";
  }
  }
 	
  if (buzzerAutoShouldBeOn || buzzerManualOverrideState) {
  if (!buzzerActive) {
	• readAndProcessSensorData(): Fungsi inti yang menjalankan pembacaan sensor dan logika kontrol utama.
	
	• Membaca suhu dan kelembaban dari sensor DHT22.
	
	• Membaca nilai analog dari sensor MQ-7 dan MQ-135, kemudian mengonversinya ke PPM menggunakan fungsi calculateRs dan calculatePPM serta nilai Ro yang telah dikalibrasi.
	
	• Menentukan buzzerAutoShouldBeOn berdasarkan ambang batas PPM CO dan polutan udara. Jika salah satu melebihi ambang batas, buzzer secara otomatis harus aktif.
	
	• Menentukan airQualityStatus ("Buruk", "Baik", atau "DHT Error").
	
	• Logika Kontrol Buzzer:
	
	  • Buzzer akan diaktifkan (HIGH) jika buzzerAutoShouldBeOn adalah true ATAU jika buzzerManualOverrideState adalah true. Ini berarti kondisi buruk akan membunyikan buzzer, dan pengguna juga bisa menyalakannya secara manual.
	
	  • Jika buzzer diaktifkan secara otomatis (bukan karena override manual), dan belum aktif, maka pin V10 di Blynk akan dikirimi nilai 1. Ini bisa digunakan untuk memicu notifikasi atau mengubah status widget di Blynk.

digitalWrite(BUZZER_PIN, HIGH);
buzzerActive = true;
if (buzzerAutoShouldBeOn &&
!buzzerManualOverrideState) {
Blynk.virtualWrite(V10, 1);
}
}
} else {
if (buzzerActive) {
digitalWrite(BUZZER_PIN, LOW);
buzzerActive = false;
}
}
 	
Serial.print("Suhu: ");
Serial.print(temperature, 1); Serial.print("
C | Kelembaban: "); Serial.print(humidity,
1); Serial.print(" % | ");
Serial.print("CO: ");
Serial.print(co_ppm, 2); Serial.print("
PPM | ");
Serial.print("AirQ: ");
Serial.print(air_quality_ppm, 2);
Serial.print(" PPM | ");
Serial.print("Status: ");
Serial.println(airQualityStatus);
}
	• Buzzer akan dinonaktifkan (LOW) jika buzzerAutoShouldBeOn adalah false DAN buzzerManualOverrideState adalah false. Jadi, untuk mematikan buzzer, kedua kondisi otomatis dan manual harus tidak mengaktifkannya.
	
	• Mencetak semua nilai sensor dan status ke Serial Monitor untuk debugging

3.4.8 Fungsi Pengiriman Data ke Blynk

Tabel 3. 8 Fungsi Pengiriman Data ke Blynk

Kode Program	Penjelasan

void sendDataToBlynk() {
Blynk.virtualWrite(V0,
temperature);
Blynk.virtualWrite(V1, humidity);
Blynk.virtualWrite(V2,
air_quality_ppm);
Blynk.virtualWrite(V3, co_ppm);
Blynk.virtualWrite(V4,
airQualityStatus);
Blynk.virtualWrite(V5,
buzzerManualOverrideState ? 1 : 0);
}
	• sendDataToBlynk(): Fungsi ini mengirimkan nilai-nilai sensor dan status kualitas udara ke Virtual Pin yang sesuai di Blynk.
	
	• V0: Suhu
	
	• V1: Kelembaban
	
	• V2: Kualitas Udara (PPM)
	
	• V3: CO (PPM)

	• V4: Status Kualitas Udara (Teks) • V5: Status manual override buzzer, memastikan widget Switch di Blynk selalu sinkron dengan status internal ESP32.

3.4.9 Fungsi Pembaruan Tampilan LCD

Tabel 3. 9 Fungsi Pembaruan Tampilan LCD

Kode Program	Penjelasan

void updateLCDDisplay() {
  lcd.clear();
 	
  lcd.setCursor(0, 0);
  if (isnan(temperature) ||
isnan(humidity)) {
  lcd.print("DHT Error!");
  } else {
  lcd.print("T:");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C H:");
  lcd.print(humidity, 1);
  lcd.print("%");
  }
 	
  lcd.setCursor(0, 1);
  lcd.print("Status:");
  lcd.print(airQualityStatus);
}
	• updateLCDDisplay(): Fungsi ini bertanggung jawab untuk menampilkan data sensor dan status di LCD.
	
	• lcd.clear(): Membersihkan layar LCD setiap kali fungsi dipanggil.
	
	• Baris pertama LCD menampilkan Suhu dan Kelembaban. (char)223 adalah karakter derajat Celsius.
	
	• Baris kedua LCD menampilkan status kualitas udara.
	
	• Ada penanganan kesalahan jika pembacaan DHT tidak valid (isnan berarti "is not a number").

3.4.10 Fungsi Setup

Tabel 3. 10 Fungsi Setup

Kode Program	Penjelasan

void setup() {
  Serial.begin(115200);
 	
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
	• setup(): Fungsi ini hanya dijalankan sekali ketika ESP32 pertama kali dinyalakan atau di-reset.

lcd.init();
lcd.backlight();
 	
lcd.print("Starting Up...");
delay(1000);
 	
dht.begin();
 	
pinMode(MQ7_PIN, INPUT);
pinMode(MQ135_PIN, INPUT);
 	
calibrateSensors();
 	
Blynk.begin(auth, ssid, pass);
 	
timer.setInterval(5000L, readAndProcessSensorData);
timer.setInterval(5000L, sendDataToBlynk);
timer.setInterval(1000L, updateLCDDisplay);
}
	• Serial.begin(115200): Menginisialisasi komunikasi serial untuk debugging pada baud rate 115200.
	
	• pinMode(BUZZER_PIN, OUTPUT) dan digitalWrite(BUZZER_PIN, LOW): Mengatur pin buzzer sebagai output dan memastikan buzzer mati di awal.
	
	• lcd.init() dan lcd.backlight(): Menginisialisasi LCD dan menyalakan backlight-nya.
	
	• Menampilkan pesan "Starting Up..." di LCD.
	
	• dht.begin(): Menginisialisasi sensor DHT.
	
	• pinMode(MQ7_PIN, INPUT) dan pinMode(MQ135_PIN, INPUT): Mengatur pin sensor MQ sebagai input.
	
	• calibrateSensors(): Memanggil fungsi kalibrasi awal untuk sensor MQ.
	
	• Blynk.begin(auth, ssid, pass): Memulai koneksi ke server Blynk menggunakan kredensial yang telah ditentukan.
	
	• timer.setInterval(...): Mengatur interval waktu untuk setiap fungsi yang akan dijalankan secara periodik:
	
	• readAndProcessSensorData dan sendDataToBlynk dijalankan setiap 5 detik (5000 milidetik).

	• updateLCDDisplay dijalankan setiap 1 detik (1000 milidetik) untuk refresh tampilan LCD lebih sering.

3.4.11 Fungsi Loop

Tabel 3. 11 Fungsi Loop

Kode Program	Penjelasan

void loop() {
  Blynk.run();
  timer.run();
}
	• loop(): Fungsi ini berjalan terus-menerus setelah setup() selesai.
	
	• Blynk.run(): Ini adalah fungsi penting dari pustaka Blynk yang harus dipanggil berulang kali. Fungsi ini menangani semua komunikasi dengan server Blynk (mengirim dan menerima data, menjaga koneksi).
	
	• timer.run(): Ini adalah fungsi dari BlynkTimer yang harus dipanggil berulang kali. Fungsi ini memeriksa apakah ada fungsi yang telah diatur untuk dijalankan pada interval tertentu dan menjalankannya jika waktunya tiba.

3.5 Pengujian Sensor

3.5.1 Sensor MQ135 dan MQ7

Tabel 3. 12 Pengujian Sensor MQ135 dan MQ7

MQ135	MQ7	Status	Buzzer	Kondisi
756	13	Baik	OFF	Status kedua sensor tidak berubah Buzzer tidak menyala
837	25	Buruk	ON	Status Sensor MQ7 berubah Buzzer menyala
1000	17	Buruk	ON	Status Sensor MQ135 berubah Buzzer menyala
1000	25	Buruk	ON	Status Kedua sensor berubah Buzzer menyala

856	20	Baik	OFF	Status kedua sensor tidak berubah Buzzer tidak menyala
553	22	Baik	OFF	Status kedua sensor tidak berubah Buzzer tidak menyala
1000	24	Buruk	ON	Status Sensor MQ135 berubah Buzzer menyala
955	25	Buruk	ON	Status Sensor MQ7 berubah Buzzer menyala
315	7	Baik	OFF	Status kedua sensor tidak berubah Buzzer tidak menyala
1000	25	Buruk	ON	Status Kedua sensor berubah Buzzer menyala

Berdasarkan data yang disajikan pada tabel 3.12 mengenai respons Sensor MQ135 dan MQ7, dapat disimpulkan bahwa penetapan status kualitas udara dan aktivasi buzzer mengikuti logika berbasis kondisi ambang batas yang mana untuk sensor MQ135 jika gas CO_2 kurang dari 1000 PPM maka statusnya Adalah “LOW” Sedangkan jika gas CO_2 lebih dari 1000 PPM maka status udara menjadi “HIGH” , dan untuk sensor MQ7 jika gas CO kurang dari 25 PPM maka statusnya Adalah “LOW” sedangkan jika gas CO lebih dari 25 PPM maka status udaranya Adalah “HIGH” Kualitas udara dikategorikan sebagai "Baik" apabila kedua sensor, baik MQ135 maupun MQ7, menunjukkan pembacaan "LOW", mengindikasikan konsentrasi polutan dan karbon monoksida berada di bawah batas toleransi yang ditentukan. Sebaliknya, status "Buruk" akan ditetapkan dan buzzer akan diaktifkan secara otomatis apabila salah satu atau kedua sensor mendeteksi pembacaan "HIGH". Hal ini mencakup skenario di mana MQ135 berada pada kondisi "LOW" sementara MQ7 "HIGH", atau MQ135 "HIGH" saat MQ7 "LOW", maupun ketika kedua sensor secara simultan berada pada kondisi "HIGH". Mekanisme ini menunjukkan bahwa sistem dirancang untuk memberikan peringatan dini terhadap adanya polutan atau gas karbon monoksida dalam konsentrasi tinggi, dengan aktivasi buzzer sebagai indikator audio kritis yang mengonfirmasi kondisi udara yang tidak aman

3.5.2 Sensor DHT22

Tabel 3. 13 Pengujian Sensor DHT22

Waktu	Suhu	Kelembaban
21.50	31.5 ℃	82.3 %
22.00	31.6 ℃	82.1 %
22.10	31.5 ℃	82.0 %
22.20	31.7 ℃	81.8 %
22.30	31.4 ℃	81.7 %
22.40	31.4 ℃	81.9 %
22.50	31.4 ℃	82.1 %
23.00	31.2 ℃	82.0 %
23.10	31.5 ℃	82.1 %
23.20	32.0 ℃	83.0 %

Berdasarkan data yang disajikan dalam Tabel 3.13 mengenai pembacaan Sensor DHT22, dapat ditarik kesimpulan bahwa selama periode pengamatan (antara pukul 21.50 hingga 23.20), kondisi suhu dan kelembaban udara menunjukkan stabilitas yang relatif tinggi. Rata-rata suhu di lingkungan pengujian berkisar antara 31.2°C hingga 32.0°C, menunjukkan kondisi yang cenderung hangat. Sementara itu, tingkat kelembaban udara tercatat dalam rentang 81.7% hingga 83.0%, mengindikasikan lingkungan yang sangat lembab. Fluktuasi nilai-nilai ini selama periode tersebut sangat minim, menunjukkan bahwa sensor DHT22 memberikan pembacaan yang konsisten dan lingkungan pengujian memiliki kondisi suhu dan kelembaban yang cukup stabil dalam rentang waktu tersebut.

3.5.3 Sensor MQ7

Tabel 3. 14 Pengujian Sensor MQ7

Waktu	Jumlah CO dalam PPM (Part Per Million) pada MQ7
21.50	9.6
22.00	9.5
22.10	9.5
22.20	9.5

22.30	9.46
22.40	9.56
22.50	8.77
23.00	9.5
23.10	9.39
23.20	9.43

Berdasarkan data yang disajikan dalam Tabel 3.14 mengenai pembacaan Sensor MQ7, dapat disimpulkan bahwa konsentrasi gas karbon monoksida (CO) di lingkungan pengujian menunjukkan fluktuasi yang sangat kecil dan cenderung stabil pada tingkat rendah selama periode pengamatan. Nilai PPM CO berkisar antara 8.77 PPM hingga 9.6 PPM. Rentang nilai ini mengindikasikan bahwa konsentrasi CO berada jauh di bawah ambang batas berbahaya bagi manusia dalam kondisi umum, dan kondisi udara relatif bersih dari kontaminasi karbon monoksida selama durasi pengujian

3.5.4 Sensor MQ135

Tabel 3. 15 Pengujian Sensor MQ135

Waktu	Jumlah CO_2 dalam PPM (Part Per Million) pada MQ135
21.50	756
22.00	537
22.10	259
22.20	300
22.30	856
22.40	553
22.50	406
23.00	955
23.10	315
23.20	750

Berdasarkan data yang disajikan dalam Tabel 3.15 mengenai pembacaan Sensor MQ135, dapat disimpulkan bahwa konsentrasi gas polutan umum, diwakili oleh nilai PPM, berada pada tingkat yang sangat rendah dan menunjukkan tren

penurunan yang gradual selama periode pengamatan. Nilai PPM yang terdeteksi berkisar antara 6.1 PPM hingga 7 PPM. Tingkat konsentrasi yang sangat rendah ini mengindikasikan bahwa kualitas udara di lingkungan pengujian sangat baik dan tidak terdapat polutan udara signifikan yang terdeteksi oleh sensor MQ135 selama waktu tersebut.

4. PENUTUP

4.1 Kesimpulan

Proyek ini telah berhasil merancang dan mengimplementasikan sebuah Prototipe Sistem Monitoring Kualitas Udara Berbasis ESP32 dengan Akuisisi Data PPM Sensor MQ-7, MQ-135, DHT22 dan Visualisasi via Blynk IoT yang memenuhi tujuan utamanya, yaitu mendeteksi dan menampilkan kondisi lingkungan udara secara real-time. Keberhasilan ini dibuktikan melalui kemampuan sistem dalam mengumpulkan data suhu, kelembaban, konsentrasi karbon monoksida, dan polutan udara umum secara berkelanjutan. Data yang terkumpul tidak hanya ditampilkan secara lokal melalui LCD, tetapi juga ditransmisikan ke platform Blynk, memungkinkan pemantauan jarak jauh yang efektif. Lebih lanjut, integrasi logika penentuan status kualitas udara ("Baik" atau "Buruk") yang diikuti dengan aktivasi buzzer sebagai peringatan audio, serta potensi notifikasi melalui Blynk, secara signifikan berkontribusi pada tujuan untuk meningkatkan kewaspadaan masyarakat terhadap bahaya polutan pada lingkungan

4.2 Saran

Berdasarkan pengembangan proyek sistem monitoring kualitas udara ini, adalah menambahkan fitur monitoring jarak jauh berbasis IoT, seperti integrasi dengan aplikasi mobile atau platform cloud (misalnya Blynk atau Firebase), sehingga pengguna dapat memantau kondisi lingkungan udara secara real-time kapan saja dan di mana saja. Selain itu, sistem dapat ditingkatkan dengan penggunaan sensor tambahan seperti sensor partikulat (PM2.5 atau PM10) atau sensor gas spesifik lainnya agar informasi polutan lingkungan lebih lengkap. Penerapan sistem notifikasi otomatis ketika terjadi gangguan, seperti peningkatan drastis level polutan melebihi ambang batas aman atau kegagalan sensor, juga akan sangat membantu dalam menjaga keandalan sistem dan meminimalkan potensi dampak negatif terhadap kesehatan.

DAFTAR PUSTAKA

[1]. Faizuddin, S., Lakshmaiah, M. V., Alam, K. T., & Ravikiran, M. (2020). ***IoT based Indoor Air Quality Monitoring system using Raspberry Pi4.*** *Proceedings of the 4th International Conference on Electronics, Communication and Aerospace Technology, ICECA 2020*

[2]. Inaldo, S. (2023). **Sistem Monitoring Pencemaran Lingkungan Hidup Berbasis *Internet Of Things* (IoT).** Jurnal Ilmiah Teknik Elektro dan Komputer, 15

[3]. Rombang, I. A., Setyawan, L. B., & Dewantoro, G. (2022). **Perancangan Prototipe Alat Deteksi Asap Rokok dengan Sistem Purifier Menggunakan Sensor MQ-135 dan MQ-2.** Jurnal Teknoif, 10

[4]. Hariyadi, D., Nakulo, B., Sari, I. D., & Aini, F. N. (2020). **Pemantauan Sistem Kualitas Udara Menggunakan OpenHAB**

[5]. Fadli, I. (2020). **Pembangunan Sistem Monitoring Kualitas Udara Dan Gas Dalam Ruangan Dengan Platform IoT Dan Notifikasi Via Android.** Jurnal FTEKNIK, 7

[6]. Prasetyo Nugroho, E., Anisyah, A., Al Fathin, D. S., et al. (2025). **Pemodelan Sistem Monitoring Kualitas Udara Pintar Berbasis Internet of Things dengan Pendekatan Machine Learning**. Jurnal Teknologi Informasi dan Ilmu Komputer (JTIIK), 12(2), 469-480.

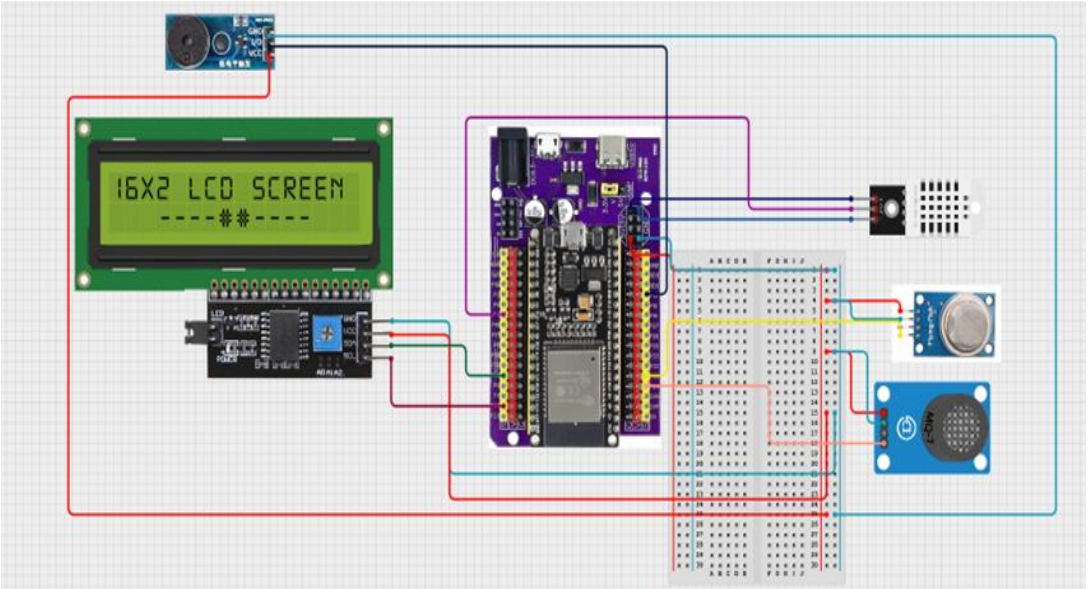
[7]. Bahar, A. P. (2021). **Rancang Bangun Alat Monitoring Polusi Udara Pada Kawasan Industri Berbasis Mikrokontroler Arduino Uno. Jurnal Teknik Elektro, 7.**

[8]. Muhammad Nadzirin Anshari Nur , Bunyamin ,dan Rosyid Wahyu Hardanang (2020), **Perancangan Sistem Keamanan Kendaraan Berbasis Internet of Things Berbasis Mikrokontroler**. Jurnal , Teknik Elektro Universitas Halu Oleo Kendari Sulawesi Tenggara

[9]. Rumampuk, G. C., Poekoel, V. C., & Rumagit, A. M. (2022). **Perancangan Sistem Monitoring Kualitas Udara Dalam Ruangan Berbasis IoT**. Jurnal Teknik Informatika, 17(1), 11-18

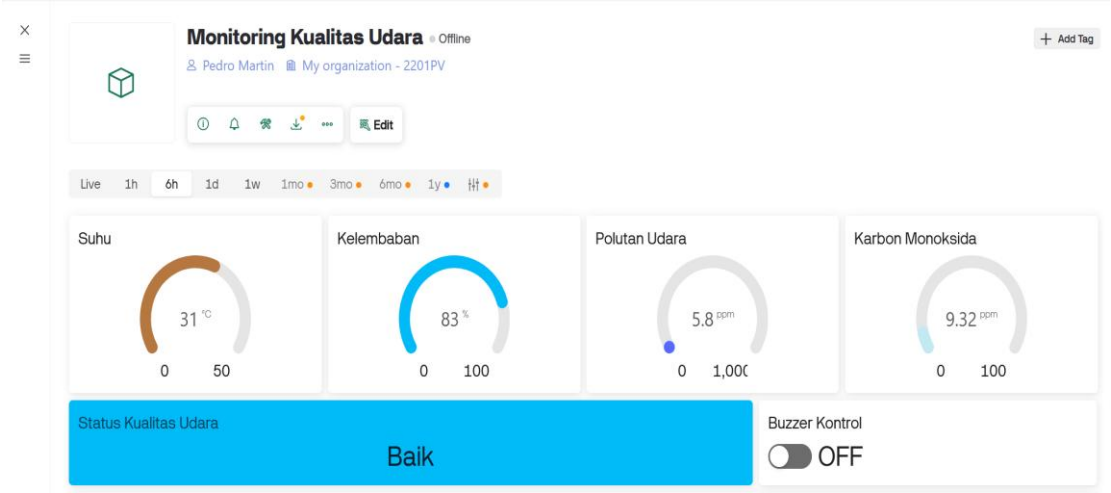
[10]. Rombang, I. A., Setyawan, L. B., & Dewantoro, G. (2022). **Perancangan Prototipe Alat Deteksi Asap Rokok dengan Sistem Purifier Menggunakan Sensor MQ-135 dan MQ-2**.

LAMPIRAN 1
GAMBAR RANGKAIAN



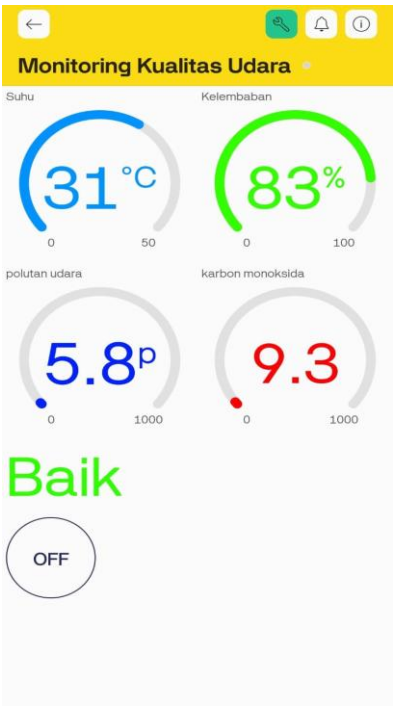
LAMPIRAN 2

TAMPILAN BLYNK PADA WEB



LAMPIRAN 3

TAMPILAN BLYNK PADA HANDPHONE



LAMPIRAN 4

GAMBAR ALAT



LAMPIRAN 5

CODE ALAT

```
#define BLYNK_TEMPLATE_ID "Isi dengan template ID anda"
#define BLYNK_TEMPLATE_NAME "Monitoring Kualitas Udara"

#define BLYNK_PRINT Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>
#include <LiquidCrystal_I2C.h>

// Kredensial WiFi Anda
char ssid[] = "Isi dengan SSID anda";
char pass[] = "Isi Dengan Password WIFI";

// Auth Token Blynk Anda
char auth[] = "Isi dengan Auth token";

LiquidCrystal_I2C lcd(0x27, 16, 2);

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

#define MQ7_PIN 34
#define MQ135_PIN 35

const float VREF = 3.3;
const float ADC_MAX = 4095.0;

const float RL_MQ7 = 10.0;
const float MQ7_CO_A = 9.87;
const float MQ7_CO_B = -0.925;
float Ro_MQ7 = 0.0;

const float RL_MQ135 = 20.0;
const float MQ135_AIR_A = 116.6020682;
const float MQ135_AIR_B = -2.769034857;
float Ro_MQ135 = 0.0;
```

```
const float CO_PPM_THRESHOLD_BAD = 25.0;
const float AIR_QUALITY_PPM_THRESHOLD_BAD = 1000.0;

float temperature;
float humidity;
float co_ppm;
float air_quality_ppm;
String airQualityStatus = "Unknown";

BlynkTimer timer;

#define BUZZER_PIN 13

bool buzzerActive = false;
bool buzzerManualOverrideState = false;

float calculateRs(int adcValue, float R_L) {
  float Vout = adcValue * (VREF / ADC_MAX);
  float Rs = (VREF / Vout - 1) * R_L;
  return Rs;
}

float calculatePPM(float rs_ro_ratio, float a_const, float b_const)
{
  if (rs_ro_ratio <= 0) return 0;
  return a_const * pow(rs_ro_ratio, b_const);
}

void calibrateSensors() {
  Serial.println("\nMemulai kalibrasi sensor MQ... Pastikan sensor
berada di udara bersih dan stabil.");
  Serial.println("Ini akan memakan waktu sekitar 10 detik...");
  lcd.clear();
  lcd.print("Kalibrasi MQ...");
  lcd.setCursor(0,1);
  lcd.print("Udara Bersih!");
  delay(10000);

  float RsSum_MQ7 = 0;
  float RsSum_MQ135 = 0;
  int numReadings = 50;

  for (int i = 0; i < numReadings; i++) {
    RsSum_MQ7 += calculateRs(analogRead(MQ7_PIN), RL_MQ7);
    RsSum_MQ135 += calculateRs(analogRead(MQ135_PIN), RL_MQ135);
```

```
    delay(100);
  }

  Ro_MQ7 = RsSum_MQ7 / numReadings;
  Ro_MQ135 = RsSum_MQ135 / numReadings;

  Serial.print("Kalibrasi selesai.");
  Serial.print(" Ro MQ-7 (CO) = "); Serial.print(Ro_MQ7, 2);
Serial.println(" kOhm");
  Serial.print(" Ro MQ-135 (Air) = "); Serial.print(Ro_MQ135, 2);
Serial.println(" kOhm");

  lcd.clear();
  lcd.print("Kalibrasi OK!");
  lcd.setCursor(0,1);
  lcd.print("Ro7:"); lcd.print(Ro_MQ7,0); lcd.print(" Ro135:");
lcd.print(Ro_MQ135,0);
  delay(2000);
}

BLYNK_WRITE(V5) {
  int pinValue = param.asInt();

  if (pinValue == 1) {
    buzzerManualOverrideState = true;
    Serial.println("Buzzer MANUAL ON dari Blynk.");
  } else {
    buzzerManualOverrideState = false; OFF
    Serial.println("Buzzer MANUAL OFF dari Blynk.");
  }
}

void readAndProcessSensorData() {
  temperature = dht.readTemperature();
  humidity = dht.readHumidity();

  float rs_mq7_current = calculateRs(analogRead(MQ7_PIN), RL_MQ7);
  float rs_ro_mq7 = rs_mq7_current / Ro_MQ7;
  co_ppm = calculatePPM(rs_ro_mq7, MQ7_CO_A, MQ7_CO_B);

  float rs_mq135_current = calculateRs(analogRead(MQ135_PIN),
RL_MQ135);
  float rs_ro_mq135 = rs_mq135_current / Ro_MQ135;
  air_quality_ppm = calculatePPM(rs_ro_mq135, MQ135_AIR_A,
MQ135_AIR_B);
```

```
  bool buzzerAutoShouldBeOn = false;
  if (co_ppm > CO_PPM_THRESHOLD_BAD || air_quality_ppm >
AIR_QUALITY_PPM_THRESHOLD_BAD) {
      buzzerAutoShouldBeOn = true;
  }

  if (isnan(temperature) || isnan(humidity)) {
      airQualityStatus = "DHT Error";
  } else {
      if (buzzerAutoShouldBeOn) {
          airQualityStatus = "Buruk";
      } else {
          airQualityStatus = "Baik";
      }
  }

if (buzzerAutoShouldBeOn || buzzerManualOverrideState) {
    if (!buzzerActive) {
        digitalWrite(BUZZER_PIN, HIGH);
        buzzerActive = true;
        if (buzzerAutoShouldBeOn && !buzzerManualOverrideState) {
            Blynk.virtualWrite(V10, 1);
        }
    }
} else {
    if (buzzerActive) {
        digitalWrite(BUZZER_PIN, LOW);
        buzzerActive = false;
    }
}

  Serial.print("Suhu: "); Serial.print(temperature, 1);
Serial.print(" C | Kelembaban: "); Serial.print(humidity, 1);
Serial.print(" % | ");
  Serial.print("CO: "); Serial.print(co_ppm, 2); Serial.print(" PPM
| ");
  Serial.print("AirQ: "); Serial.print(air_quality_ppm, 2);
Serial.print(" PPM | ");
  Serial.print("Status: "); Serial.println(airQualityStatus);
}

void sendDataToBlynk() {
  Blynk.virtualWrite(V0, temperature);
  Blynk.virtualWrite(V1, humidity);
```

```
  Blynk.virtualWrite(V2, air_quality_ppm);
  Blynk.virtualWrite(V3, co_ppm);
  Blynk.virtualWrite(V4, airQualityStatus);

  Blynk.virtualWrite(V5, buzzerManualOverrideState ? 1 : 0);
}

void updateLCDDisplay() {
  lcd.clear();

  lcd.setCursor(0, 0);
  if (isnan(temperature) || isnan(humidity)) {
    lcd.print("DHT Error!");
  } else {
    lcd.print("T:");
    lcd.print(temperature, 1);
    lcd.print((char)223);
    lcd.print("C H:");
    lcd.print(humidity, 1);
    lcd.print("%");
  }

  lcd.setCursor(0, 1);
  lcd.print("Status:");
  lcd.print(airQualityStatus);
}
void setup() {
  Serial.begin(115200);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  lcd.init();
  lcd.backlight();

  lcd.print("Starting Up...");
  delay(1000);

  dht.begin();

  pinMode(MQ7_PIN, INPUT);
  pinMode(MQ135_PIN, INPUT);

  calibrateSensors();
```

```
  Blynk.begin(auth, ssid, pass);

  timer.setInterval(5000L, readAndProcessSensorData);
  timer.setInterval(5000L, sendDataToBlynk);
  timer.setInterval(1000L, updateLCDDisplay);
}

void loop() {
  Blynk.run();
  timer.run();
}
```

LAMPIRAN 6

BIAYA PEMBUATAN ALAT

NO	Komponen	Jumlah	Harga satuan (Rp)	Total harga (Rp)
1	Sensor MQ135	1	16.000	16.000
2	Sensor MQ7	1	25.000	25.000
3	Sensor DHT22	1	21.000	21.000
4	ESP32	1	65.000	65.000
5	Buzzer	1	8.000	8.000
6	Jumper	3	10.000	30.000
7	Electro Box	1	33.000	33.000
8	LCD 16x2	1	26.000	26.000
Total Biaya				224.000